Hybrid Adaptive Consensus: A Sovereign Finite-State-Machine Fallback for Modular Blockchains

Author One[1] Author Two[1]

[1]Department Name, Affiliation/University, City, Country

email1@domain.com, email2@domain.com

Abstract

Modular blockchains obtain settlement and data availability (DA) from independently operated layers. A chain whose block validity requires Bitcoin settlement receipts and DA receipts inherits the liveness of those layers: when either stalls, no proposal satisfies the validity predicate and the chain halts, although its validators are connected and correct. Dropping the requirement restores progress but suspends the guarantees it provided. We present *Engram Hybrid Adaptive Consensus*, a protocol layer on a CometBFT agreement core that makes this suspension an agreed, deterministic state transition. Proposer selection, voting, quorum intersection, and deterministic finality remain CometBFT's. Engram adds (i) a *health certificate* – a deterministic aggregate of the height-bound, signed peripheral observations of $\geq 2f+1$ voting power, carried as vote extensions and bounded by the range of honest observations (Lemma 1) – and (ii) a four-state finite state machine (ANCHORED, SUSPICIOUS, SOVEREIGN, RECOVERING) evaluated deterministically on that certificate. When Bitcoin or DA degrades, or the peripheral P2P view degrades while the core consensus network retains quorum connectivity, the chain keeps producing blocks in SOVEREIGN mode under the validator quorum, relaxes the receipt requirement of the unhealthy layer, and locks cross-chain withdrawals. It returns to ANCHORED after recovery hysteresis, DA backfill of the SOVEREIGN interval, and a validator-verified zero-knowledge recovery proof (Noir/UltraHonk, 256 headers per proof, chained for longer intervals) that attests to header continuity, FSM-transition legality, and withdrawal-lock adherence over the interval. We give a layered TLA+ specification and check each layer in isolation with bounded model checking (zero violations at $N \leq 4$; the largest run, the FSM safety check, explored 3.39 million distinct states) with a five-mechanism ablation study in which each removed mechanism reproduces a distinct predicted failure; the cross-layer refinement obligation remains open. On a 7-validator Cosmos SDK testnet with live Bitcoin regtest and Celestia networks, under a layered composite fault the chain keeps committing blocks and completes a full SOVEREIGN→RECOVERING→ANCHORED cycle with 100% AppHash agreement. The extended consensus object adds +228 bytes per proposal and 18.25 μs of validation per block, yet lowers the block-production rate by roughly 20% relative to vanilla CometBFT. Scanning every height shows that this is not extra consensus rounds: all heights commit in the first round, with no nil precommit and no rejected proposal. Every block instead takes a near-constant 0.24 s longer (1.27 s against 1.03 s). [PENDING: add the hard-gated-baseline stall result.]

Keywords: blockchain consensus, Byzantine fault tolerance, modular blockchain, finite state machine, formal verification, TLA+, zero-knowledge proofs

1. Introduction

Modular blockchains separate execution from services such as settlement and data availability (DA), which independently operated networks provide. Consider a chain whose validity predicate requires

fresh evidence from such layers: a block is valid only if it passes the BFT checks and carries both a Bitcoin settlement receipt and a receipt from a DA network such as Celestia [1], each attesting a height within a tolerance τ of the block. Formally,

$$\mathit{valid}(B) = \mathit{BFTvalid}(B) \wedge \mathit{BTCreceipt}(B) \wedge \mathit{DAreceipt}(B).$$

If Bitcoin settlement is congested or censored, or the DA network is unavailable, no proposal satisfies the predicate. The BFT consensus layer [2, 3] built on top stalls although the validator set is fully connected and correct. The chain inherits the liveness of the layers it depends on.

Dropping the requirement restores progress but also suspends the guarantees that motivated the modular design. A chain that proceeds without a settlement checkpoint cannot claim the same externally anchored settlement guarantee, and a chain that proceeds without DA confirmation cannot claim that its recent data is externally available. The problem is therefore not only to detect an outage. The protocol must make all validators agree on when an external guarantee may be suspended, restrict the operations whose safety depended on it, and define the evidence required before the guarantee is restored. Engram keeps the chain live by suspending the external dependency together with the operations whose safety relied on it.

The gap. Existing work addresses parts of this problem under other failure models. Protocols that trade finality for availability adapt to validator participation or consensus-network conditions, not to external infrastructure [4, 5]. Rollup mechanisms provide alternative paths when sequencing or data publication fails, but the decision rests with users, a sequencer, a committee, or a contract on the parent chain [6–8]. Oracle designs and ABCI++ vote extensions let validators bring externally observed values into replicated state [9, 10], but the value goes to the application. None of them covers a native chain whose validity depends on independently operated external layers that different validators observe differently. There, validators must agree on the observation and on the change in safety policy that follows from it (Section 2).

Why the decision belongs in consensus. The relaxation of receipt requirements and the withdrawal circuit breaker change two things that every validator must agree on: whether a block is valid, and whether a withdrawal executes. Bitcoin, DA, and peripheral-network status are observed locally and asynchronously, so two honest validators may see different conditions at the same instant. Suppose each validator applied a local policy to its own view. Validators would disagree on validity, so proposals would collect nil votes and rounds would repeat, or they would execute withdrawals differently and produce diverging `AppHash` values. A local, non-deterministic observation cannot drive replicated state directly. It must first become a deterministic input that every validator evaluates identically.

Design approach. Engram keeps CometBFT's proposer selection, prevote/precommit protocol, quorum intersection, and deterministic finality unchanged, and modifies the object and the validity policy that this pipeline carries. At height $h-1$, each validator signs its sensor snapshot as a vote extension; the sensors are read only there. The proposer of height h includes the signed snapshots of at least $2f+1$ voting power in a *health certificate* C^h. Every validator verifies C^h, recomputes a deterministic weighted-median/majority aggregate $\mathsf{Agg}(C^h)$, and accepts a proposal only if its target state satisfies $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$, where δ is the transition function of a four-state finite-state machine (FSM) with states Anchored, Suspicious, Sovereign, and Recovering. FSM state is written only from a certificate that CometBFT has already agreed on (Section 3.4). "Hybrid Adaptive Consensus" denotes this adaptive layer inside the consensus pipeline, not a replacement for the agreement protocol.

The FSM makes the chain's security mode explicit. In ANCHORED, blocks require the Bitcoin and DA receipts and withdrawals are permitted. Sustained degradation moves the chain through SUSPICIOUS into SOVEREIGN, where the validator quorum keeps ordering blocks, the receipt requirement of each unhealthy layer is relaxed, and cross-chain withdrawals are locked. When the layers recover, the chain enters RECOVERING. Return to ANCHORED requires recovery hysteresis, backfill of the SOVEREIGN interval's data to the DA layer, and a validator-verified zero-knowledge proof that the interval is hash-linked, obeyed the FSM transition rules, and respected the withdrawal lock (Sections 3.5 and 3.9; the prototype's proof gate is qualified in Section 7.3).

Scope. Engram handles degradation of Bitcoin, Celestia, and the peripheral P2P interfaces while the CometBFT quorum remains live. It does not address loss of the core quorum: like any BFT protocol, it needs at least $2f+1$ validators to communicate after global stabilization time. "P2P degradation" in this paper always means partial, peripheral degradation (Section 3.2).

To our knowledge, no published design combines the following for a modular chain whose validity depends on external settlement or DA layers: (i) a quorum-backed set of validator observations from which every validator deterministically derives external-layer health; (ii) deterministic adaptation of both block-validity requirements and withdrawal admissibility from that health while the native quorum keeps ordering blocks; and (iii) a recovery condition that requires a proof over the degraded interval and its DA backfill. Engram targets this combination.

Contributions.

- **Quorum-backed external-health agreement.** A height-bound certificate of signed observations from at least $2f+1$ voting power, carried in vote extensions, and a deterministic weighted-median/majority aggregation rule. Under $f < n/3$, every numeric aggregate stays within the range of the honest observations in a well-formed certificate, whichever observations a Byzantine proposer includes or omits (Section 3.4).
- **Dependency-aware adaptive execution.** A four-state FSM on the certificate, with asymmetric hysteresis and exponential backoff on repeated failed recovery, that changes the Bitcoin and DA receipt requirements without modifying CometBFT's agreement core. A withdrawal circuit breaker, following the circuit-breaker resilience pattern [11], locks cross-chain withdrawals while the chain runs without external corroboration (Sections 3.5–3.7).
- **Proof- and backfill-gated recovery.** A Noir/UltraHonk circuit that proves header-chain continuity, FSM-transition legality, withdrawal-lock adherence, and boundary-validator signatures for up to 256 SOVEREIGN-interval headers at constant proving and verification cost. Longer intervals use a sequence of proofs linked through an on-chain committed root, optionally folded by an aggregator (Appendix C). The proof shows that the interval is valid; the exit condition also requires that its data was published to the DA layer. Validators verify each proof inside block execution; Bitcoin timestamps the resulting checkpoint but does not verify the proof (Section 3.9).
- **Layered formal specification.** A four-layer TLA+ specification. Bounded model checking of each layer's safety and liveness properties found zero violations at $N \leq 4$; the largest single run explored 3.39 million distinct states. In a five-mechanism ablation study, removing each mechanism reproduces a distinct predicted counterexample. The cross-layer refinement obligation is specified but open (Section 4).
- **Live evaluation.** A 7-validator Cosmos SDK testnet against live Bitcoin regtest and Celestia networks with network- and application-level fault injection, comparing Engram against a

hard-gated modular baseline and a vanilla CometBFT control (Sections 5–6).

Table 1: Research Questions

RQ	Question
RQ1	**Safety.** Under up to f colluding Byzantine validators, does certifying peripheral health by quorum prevent conflicting FSM states, forged or stale receipts, false health claims, and withdrawal leakage?
RQ2	**Liveness.** When Bitcoin or DA becomes unavailable, or the peripheral P2P view degrades while core quorum connectivity holds, does Engram keep committing blocks where a hard-gated modular baseline stalls?
RQ3	**Recovery.** Once peripherals recover, do hysteresis and the recovery proof return the chain to a stable ANCHORED state without oscillation?
RQ4	**Cost.** What overhead do the extended consensus object, certificate validation, and recovery proof add, and where does it come from: proposal size, validation time, or additional consensus rounds?

The remainder of this paper is organized as follows. Section 2 situates Engram in the literature. Section 3 presents the protocol, including the health certificate and the recovery proof. Section 4 presents the formal specification and model-checking results. Section 5 describes the testbed and baselines, Section 6 answers RQ1–RQ4, Section 7 discusses limitations, and Section 8 concludes.

2. Related Work

A chain whose block validity depends on external layers raises five questions, and each has been studied by a separate community. Fig. 1 organizes the literature by these questions: how consensus behaves under degraded conditions (Section 2.1), how local observations of an external layer become data that all validators see and agree on (Section 2.2), how modular designs cope with the failure of Bitcoin, DA, or settlement layers (Section 2.3), how a chain returns to the anchored state (Section 2.4), and how such designs are validated (Section 2.5). Several building blocks of Engram exist in prior work: vote extensions that carry signed external data, weighted-median aggregation, and protocol-defined fallback for DA and sequencing failures. Engram differs in what the aggregated observation controls. Table 2 compares representative systems by trigger, decision authority, and behavior under failure, and Section 2.6 states the combination that the comparison leaves open.

2.1. Consensus Under Degraded Conditions

Agreement core. PBFT [50] and the partial-synchrony model of Dwork, Lynch, and Stockmeyer [51] underlie Tendermint and its CometBFT successor [2, 3], which provide deterministic finality with a rotating proposer. HotStuff [52] achieves responsiveness and linear communication under a correct leader within the same leader-based lineage. Narwhal/Tusk [53] separate transaction dissemination from ordering, and Bullshark [54] orders transactions on top of the Narwhal DAG. These protocols agree on the content of a proposal; whether a proposal is acceptable is decided by an application predicate that every correct validator must evaluate identically. Tendermint's consensus problem includes such a `valid()` predicate [3], and Cachin et al. [55] introduced the *external validity* condition for multi-valued Byzantine agreement. Engram relies on the same structure: its proposal-validation predicate depends on the health certificate, so the predicate stays deterministic only if the certificate is itself agreed on. BFT forensics [56] studies how many malicious replicas can be identified after a safety violation. AdoB [57] mechanizes benign and Byzantine consensus in one

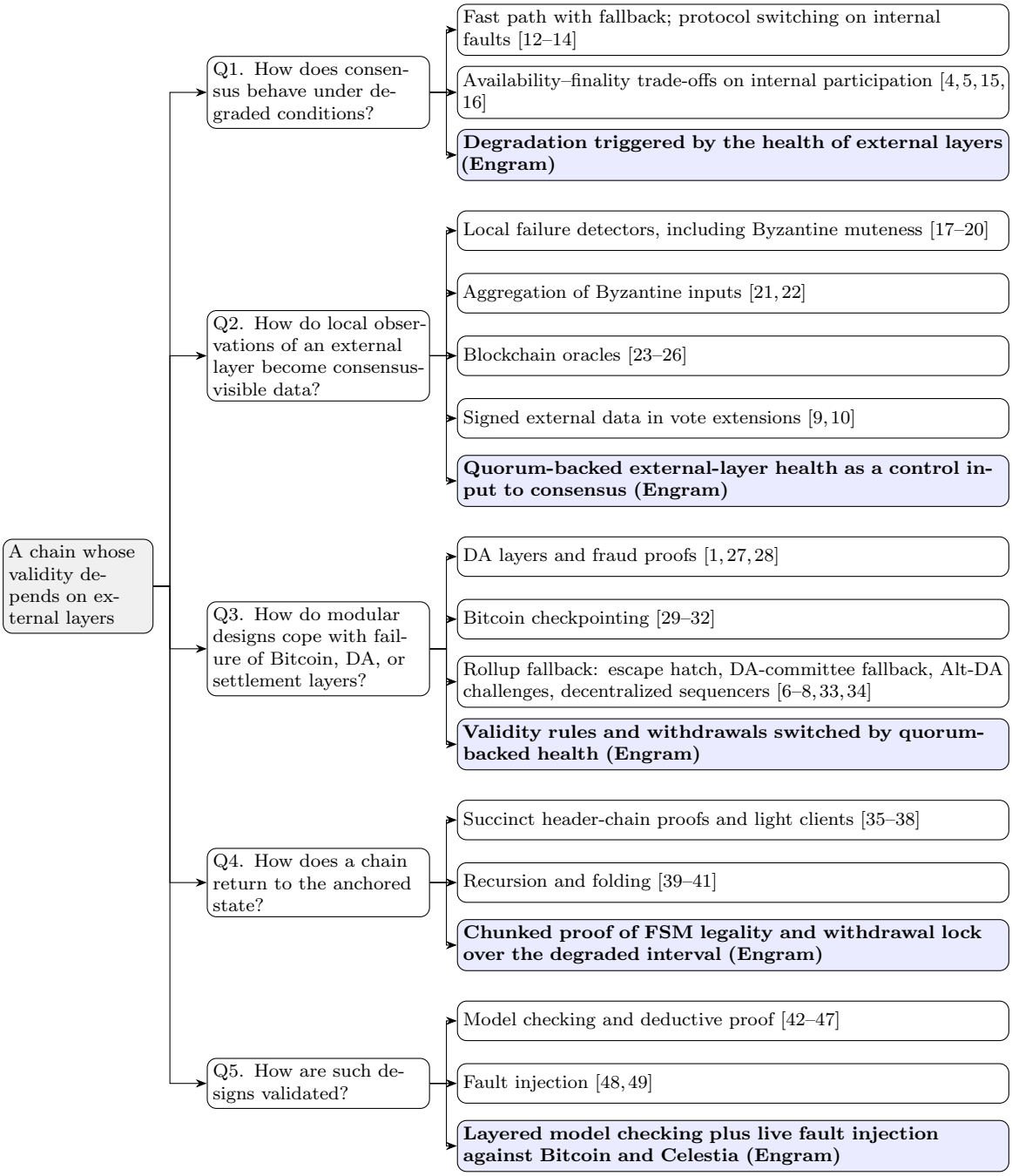


Figure 1: Taxonomy of related work, organized by the question each line of work answers. Shaded leaves mark where Engram sits. Vote-extension oracles supply signed external data to consensus (Q2) but attach no fallback semantics to it.

model, and Adore [58] adds certified reconfiguration. Engram inherits CometBFT's evidence pool for accountability and leaves the agreement core unchanged.

Fast paths and protocol switching. A second group changes the protocol's behavior in response to conditions that the replicas themselves observe. Abstract [13] lets a replicated state machine abort a client request when something goes wrong, which supports composing and reconfiguring BFT protocols. Thunderella [12] combines an optimistic fast path with a synchronous fallback, and Autobahn [14] targets seamless recovery from network blips. The trigger in each case is a fault of the replicas or of the network among them.

Availability–finality trade-offs. The closest conceptual relatives give up finality to keep the chain growing. The sleepy model [15] lets players be online or offline and gives a consensus protocol that is resilient as long as a majority of the online players are honest. Ebb-and-flow protocols [4] expose both a dynamically available ledger and a finalized prefix, so the chain keeps growing when finality stalls and the finalized ledger catches up on recovery. Casper FFG [16], and Gasper [5], which combines it with the LMD GHOST fork-choice rule, prove accountable safety and plausible liveness. For the case in which more than a third of the validators crash, Casper FFG proposes an inactivity leak that drains the deposits of non-voting validators until the voting validators again form a supermajority, and leaves the exact formula open. These designs share the "degrade, do not halt" goal and the tension described by the CAP theorem [59]. They differ from Engram in two respects: the trigger is *internal* participation, and the response changes how finality is reached, not which blocks and withdrawals are valid.

2.2. From Local Observations to Consensus-Visible External Data

Failure detectors. Unreliable failure detectors [17], their QoS metrics [18], and accrual detectors [19] give each process a local, eventually accurate suspicion of other processes. Muteness failure detectors [20] extend the abstraction to Byzantine processes that stop sending the messages an algorithm expects while possibly continuing to send others. Engram's sensors play the same role for peripheral layers, with two differences: a local suspicion has no effect until a quorum of signed observations backs it, and the adversary controls up to f of the reporters.

Aggregating Byzantine observations. Approximate agreement [21] requires non-faulty processes to decide on values that are close together rather than identical. Median validity [22] requires the decision to be close to the median of the non-Byzantine inputs and, at maximal fault tolerance with $n \geq 3t + 1$, within the range of those inputs. The Engram health certificate satisfies an analogous range-validity property for each weighted-median metric. It does not run a second approximate-agreement protocol: validators sign individual height-bound sensor snapshots carried as vote extensions, CometBFT agrees on the proposal that carries them, and every validator then recomputes the aggregate deterministically (Section 3.4).

Blockchain oracles. Oracle research addresses how replicated applications import external information. Town Crier authenticates web data with trusted hardware [23]. A systematization of oracle designs classifies their trust assumptions and manipulation risks [24]. DecenTruth combines multi-source truth discovery with decentralized consensus to tolerate dishonest reports [25], and PSCBO analyzes attacks on consensus-based oracle designs [26]. In all of these, the oracle supplies a value to the application layer.

Vote extensions. The closest mechanism to Engram's certificate is the ABCI++ vote extension. Each validator extends its precommit with application data, which may be obtained non-

deterministically, and the proposer of the next height receives the signed extensions and includes them in its proposal. The Cosmos SDK design record requires signatures from at least two thirds of the voting power for the proposal to be valid and uses an oracle price fetcher as its example [9]. Slinky applies this pattern to off-chain price data and aggregates the extensions with a stake-weighted median [10]. Engram uses the same transport: sensor snapshots taken at height H reach the proposer at $H+1$, where the deterministic aggregation of at least $2f+1$ voting power's worth of signed snapshots forms the health certificate. The difference lies in what the aggregate controls. A price oracle supplies a value to the application. The health certificate describes the infrastructure on which block validity depends, and it determines the receipt requirements, the withdrawal policy, and the entry into and exit from a degraded mode.

Peripheral P2P attacks. Sybil attacks [60], in which one entity presents multiple identities, and eclipse attacks on Bitcoin's P2P layer [61, 62], which monopolize a node's connections, let an adversary control what a node sees of an external network. They motivate the multi-signal peer-health profile that Engram's P2P sensor uses and are one reason a single local reading is not trusted.

2.3. Fallback for Modular-Chain Dependencies

Data availability. LazyLedger/Celestia [1] decouples DA from execution. Fraud and data availability proofs [27] let light clients detect withheld data, and coded distributed arrays target the peer-to-peer layer that distributes data availability sampling fragments [28]. This line of work makes DA *verifiable*. A chain that requires a DA inclusion proof before accepting a block still stops when DA stops. Engram's DA sensor is independent of the sampling design; what Engram adds is a consensus-level response to DA unavailability and a backfill requirement before the chain returns to ANCHORED (Section 3.9).

Bitcoin anchoring. Pegged sidechains [63] introduced two-way pegs to Bitcoin. Karakostas and Kiayias [29] study checkpointing as a defense of proof-of-work ledgers against 51% attacks and report a liveness attack, "block lead", against existing checkpointing designs. For proof-of-stake chains, Babylon [30, 32] and Pikachu [31] checkpoint onto Bitcoin to prevent long-range attacks. Tas et al. [32] show that slashable safety is impossible without an external source of trust and state their guarantees under the assumption that Bitcoin is safe and live. Their design also defines a fallback in which Bitcoin orders the PoS blocks after censorship, which strengthens the PoS chain's dependence on Bitcoin. To our knowledge, none of these publications specifies how the chain operates while the anchor itself is unreachable. Engram treats unreachability as an input, backs it with a quorum of signed observations, keeps ordering blocks, and locks withdrawals while it persists. Engram does not implement Bitcoin-enforced slashing; accountability comes from CometBFT's equivocation evidence.

Rollup fallback and its decision locus. When a rollup operator stops building blocks or proposing state roots, escape hatches [6] let users withdraw on the L1 by proving their assets against an L2 state root. Arbitrum [64] resolves disputes about contract behavior with a challenge-based protocol on the L1. Surveys of layer-two protocols [65] treat the ability of honest users to withdraw as a core security property, and list among the open weaknesses that layer-two designs assume the underlying blockchain eventually accepts transactions. Capretto et al. [33] give fraud-proof games, mechanized in Lean 4, with which an honest agent can prove on the L1 that a sequencer-and-DAC "arranger" violated a safety property even if all of its replicas are compromised; a liveness violation that preserves safety cannot be proved on the L1. A follow-up decentralizes the sequencer and the DA committee with set consensus [34].

Two designs define a protocol-level response to a DA failure. AnyTrust normally relies on a data-availability committee. If the sequencer cannot collect enough committee signatures within a few minutes, it posts the full data to the parent chain as a non-AnyTrust rollup would do [7]. Withdrawals then incur the delay of a standard rollup, and the chain switches back to the cheaper mode when the committee resumes [66]. The sequencer makes this decision, and the failure it reacts to is a committee failure. OP Stack Alt-DA keeps data with an external DA provider and lets anyone challenge a commitment on the L1. If the data is not posted before the resolve window elapses, derivation reorgs the L2 blocks that depended on it [8]. This design answers the same question as Engram, what happens when an external DA input fails, by resolving the failure on the L1 and discarding the affected blocks.

These designs show that protocol-defined fallback for DA and sequencing failures exists, and that the sequencer can be decentralized. They differ from Engram in what is agreed, by whom, and what the agreement controls. The decision rests with a sequencer, a committee, users, or L1 contracts. The observed failure is a committee or data-posting failure, not the health of Bitcoin settlement or of an external DA network. The response either publishes more data or reorganizes the chain, not a change in the validity predicate and the withdrawal policy. Engram keeps the decision inside the validator set's own consensus instance, bases it on signed observations of all three peripheral layers, and commits the FSM state in the same block.

Withdrawal risk. Lee et al. [67] break a bridge into its components, identify the risk each introduces, and analyze past exploits that targeted bridges. They report that over USD 1 billion in digital assets was stolen, incorrectly minted, or locked in bridges in the year before their study. Withdrawals are the path through which a chain without external corroboration can lose assets, which is why Engram locks them while the chain runs in SOVEREIGN mode.

2.4. Recovery and Re-Anchoring

Succinct proofs are the standard tool for convincing a verifier of a chain's history without re-executing it. They differ in how a long history is covered. Mina [37] keeps a constant-size proof of the whole chain by recursively composing SNARKs, which Plumo's authors report imposes a large overhead on the protocol being proved [36]. zkBridge [35] verifies source-chain consensus inside a two-layer recursive proof system. Plumo instead syncs a client through a chain of sequential SNARK transition proofs, each covering four months of blockchain history [36]. Recursive composition [39, 40] and folding [41] are the general techniques behind the recursive designs. FlyClient [38] takes a different route for proof-of-work chains: a light client downloads only a logarithmic number of block headers, selected by probabilistic block sampling. All of these prove that a header chain, or a validator-set transition, is valid. A chain that has run without external corroboration must also show that the interval followed the protocol. The long-range attack literature [68] reviews how an attacker can rewrite the history of a proof-of-stake chain, which is the exposure of an interval secured only by the chain's own validators. Engram's recovery proof is a fixed-capacity PLONK-family [69] proof over one interval chunk, and chunk proofs are chained through an on-chain root, as Plumo chains its transition proofs. The statement differs: it covers FSM-transition legality and withdrawal-lock adherence in addition to header continuity (Section 3.9).

2.5. Validation Methodology

TLA+ [70] with TLC and Apalache [42] has been used to specify and model-check Tendermint's blockchain synchronization protocol [43]. Deductive approaches establish properties by machine-

Table 2: Failure-handling semantics of representative systems, assessed from each design's primary publication or specification. Each row states what triggers the response, which observation supports it, who decides, and what happens to execution, withdrawals, and recovery. "n/a" marks a column the design does not address.

System	Trigger	Observation	Decision authority	Degraded behavior	Withdrawals	Recovery
Ebb-and-flow [4]	Internal participation or partition	Consensus participation	Validator protocol	Available ledger keeps growing; finalized prefix lags	n/a	Finalized prefix catches up
Casper FFG leak [16]	Loss of $> \frac{1}{3}$ voting participation	Missing votes	Validator protocol	Non-voting stake is drained	n/a	Voting validators regain a supermajority
Vote-extension oracle [9, 10]	External data update	Signed vote extensions, stake-weighted median	Validator consensus pipeline	No fallback mode	Application-defined	Continuous
Babylon [32]	PoS censorship or stall	Bitcoin checkpoints and liveness transactions	Clients and Bitcoin[a]	Bitcoin orders PoS blocks	n/a	n/a
Escape hatch [6]	Operator stops	L1 state root	Users	Exit through L1	Users withdraw through L1	n/a
AnyTrust [7, 66]	Too few committee signatures	DAC certificate and timeout	Sequencer	Full batch posted to L1	Standard rollup delay	Committee path resumes
OP Alt-DA [8]	External DA input unavailable	L1 challenge	L1 contract and derivation rules	Data resolved on L1, or affected L2 blocks reorged	Standard rollup	Challenge resolution
Engram	Bitcoin, DA, or peripheral-P2P health	Quorum of signed validator observations	Validator BFT pipeline	Receipt requirements relaxed for the failed layer	Locked in SOVEREIGN and RECOVERING	Proof and DA-backfill gate, hysteresis[b]

Note. (a) The emergency break and the fallback to Bitcoin are client-side rules, not validator consensus. The guarantees assume that Bitcoin is safe and live. AnyTrust cells follow its public documentation, not a peer-reviewed paper. (b) The DA-backfill condition is enforced in the prototype. The proof condition can be satisfied by a heuristic unless proof-only recovery is enabled (Section 7.3).

checked proof instead of by exploring a bounded instance. Ivy [44] verifies safety of infinite-state systems interactively, and Lotan and Shoham [71] automate liveness proofs in first-order logic over finite but unbounded domains. IronFleet [45] and Verdi [46] verify distributed-system implementations, the latter in Coq, and LiDO [47] gives mechanized safety and liveness proofs in Coq for unpipelined and pipelined Jolteon. A recent preprint places these approaches on a maturity scale [72]. Engram sits in the model-checking family, with a per-layer result and an open refinement obligation (Section 4). On the empirical side, Twins [48] tests BFT implementations by duplicating validator identities, and chaos engineering [49] injects faults into production systems. Engram's evaluation combines network-level and application-level fault injection against live Bitcoin and Celestia networks.

2.6. Research Gap

Prior work supplies the following elements: consensus that adapts to internal participation faults, oracle systems that aggregate external observations, vote extensions that deliver signed external data to the validator consensus pipeline, rollup mechanisms that fall back when sequencing or data

availability fails, and succinct proofs that authenticate chain histories. When an external layer fails, a chain that gates validity on that layer halts, and a chain that drops the dependency gives up the settlement and availability guarantees that motivated it. The fallbacks surveyed above place the decision with users, watchers, a sequencer, a committee, or an L1 contract.

To the best of our knowledge, no published design combines (i) a quorum-backed set of height-bound validator observations from which every validator deterministically derives the health of external layers, (ii) use of that health inside the validator consensus pipeline to change both proposal-validity requirements and withdrawal admissibility while the native validator quorum keeps ordering blocks, and (iii) a recovery condition that requires a proof over the degraded interval and the backfill of that interval to the DA layer before the chain returns to the externally anchored mode. Engram provides these three, with the proof-gate limitation of Section 7.3, retains CometBFT as the agreement protocol, and Sections 3–6 evaluate the combination. The adaptive-fallback idea first appeared as a sketch in an unpublished whitepaper [73]; the storage and slashing layers of that system are outside this paper.

3. Engram Hybrid Adaptive Consensus

3.1. Architecture Overview

Engram is a Cosmos SDK application [74] running on a CometBFT core. Three peripheral layers are monitored: (i) Bitcoin, used as a settlement and finality anchor via periodic checkpoints; (ii) Celestia [1], used as the DA layer; and (iii) the peer-to-peer interfaces through which validators reach the Engram, Bitcoin, and Celestia networks. Engram follows one rule throughout: *sensors observe, the quorum certifies, the state machine decides.* Each validator's sensors read the peripheral layers locally and asynchronously, so two honest validators may observe different values at the same instant. Those readings never enter replicated state directly. Instead, each validator signs its reading as a CometBFT vote extension; the next proposer aggregates a quorum of them into a *health certificate* (Section 3.4); and the FSM transition is a deterministic function of that certificate and the previously committed FSM state (Fig. 2).

3.2. Threat Model

Validators and network. We assume partial synchrony: message delay is unbounded before an unknown global stabilization time (GST) and bounded after it [51]. An adversary controls up to $f = \lfloor (n-1)/3 \rfloor$ of n validators by voting power (f=2 at n=7); Byzantine validators may deviate arbitrarily and collude. The adversary cannot forge signatures, find hash collisions, or subvert the UltraHonk backend's KZG setup. After GST, the *core consensus network* connects at least $2f+1$ voting power, as CometBFT itself requires.

Peripheral layers. Bitcoin, Celestia, and the peripheral P2P interfaces may be slow, unavailable, partitioned from some validators, or adversarially degraded (delayed, censored, eclipsed) for arbitrarily long. They are not assumed to be Byzantine-fault-tolerant from the validator set's perspective, but Bitcoin reorganizations deeper than a configurable depth $K_{\text{DEEP_FINALITY}}$ are excluded, as in every Bitcoin-anchoring design (Section 2). Honest validators' peripheral observations may differ from one another at any instant; we do *not* assume identical views. Instead, correctness rests on the health certificate (Section 3.4), which is sound under arbitrary honest disagreement and makes transitions deterministic.

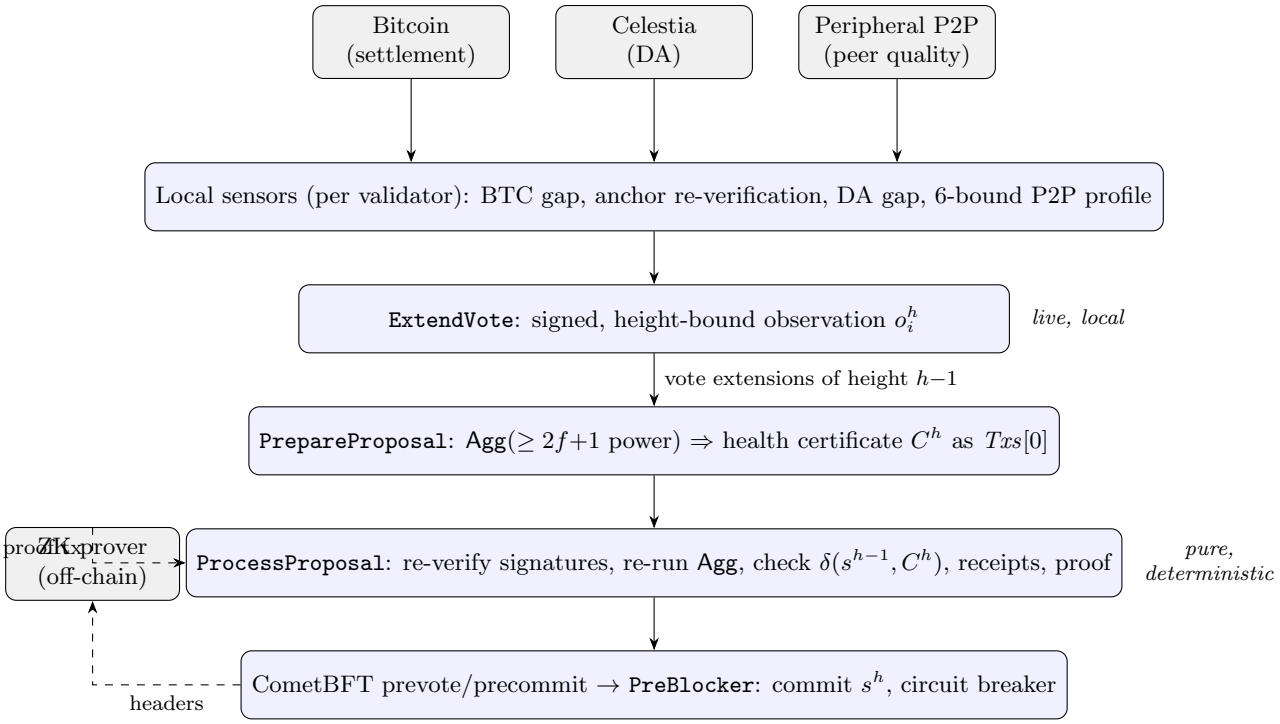


Figure 2: Engram architecture. Live, validator-local sensor reads (top) enter consensus only as signed vote extensions; everything from certificate aggregation downward is a pure function of data every validator holds, so the committed FSM state s^h is deterministic. The dashed path is the recovery proof (Section 3.9).

Parameters. FSM thresholds, hysteresis limits, and $K_{\text{DEEP_FINALITY}}$ are genesis parameters shared by every validator, like the block time or quorum size. A validator running different parameters simply fails to validate honest proposals; this affects only its own participation, not safety.

Layered adversary. The adversary may degrade the peripheral connectivity of *every* honest validator at once (delay Bitcoin headers, cut the DA bridge, eclipse the peripheral P2P interface), but it cannot partition the core consensus network below $2f+1$ voting power. Degrading the periphery therefore changes which FSM state the chain is in; it does not by itself stop block production.

More than f Byzantine validators. Beyond f, no BFT protocol retains agreement. In Anchored and Suspicious, a checkpoint that carries the state root lets any Bitcoin observer detect a fork, because two conflicting histories cannot both match a Bitcoin-timestamped root; this is accountability, not prevention. In Sovereign nothing outside the validator set constrains the chain, which is why withdrawals are locked in that state (Section 3.3).

Security of mode switching. A transition happens only when a well-formed health certificate, built from at least $2f+1$ voting power, implies it (Lemma 1, Section 3.4). A coalition of at most f validators therefore cannot force or block a transition. The remaining attack surface is griefing by an adversary that degrades the peripheral view of a quorum (Section 7).

3.3. Security Semantics by Mode

We state what each FSM state guarantees, using four properties of the committed chain $B_1, \ldots, B_h$. A *prefix* $B_1, \ldots, B_c$ is the chain up to height $c \leq h$.

- P_{agree}: no two conflicting blocks are committed at the same height. Assumption: $f < n/3$.
- $P_{\text{settle}}(c)$: the prefix up to c cannot be rewritten without a Bitcoin reorganization deeper than $K_{\text{DEEP_FINALITY}}$. Assumptions: such reorganizations are excluded (Section 3.2), and the Bitcoin checkpoint that buries height c commits the state root at c.
- $P_{\text{avail}}(c)$: the block data of every height up to c is retrievable from the DA layer. Assumption: the DA layer is honest and the data was published.
- P_{exit}: a withdrawal commits only against a chain whose settled and available prefix is the one the withdrawal depends on, up to the receipt tolerance defined below. It is enforced by the circuit breaker.

Table 3 lists which properties hold in each state.

Table 3: Properties by FSM state. † holds only if the checkpoint commits the state root and the recovery exit gate includes DA backfill (Section 3.9). The prototype implements both; the proof half of the exit gate is subject to the limitation in Section 7.3.

State	P_{agree}	P_{settle}	P_{avail}	**Withdrawals**
ANCHORED	Yes ($f < n/3$)	Yes†	Yes†	Permitted
SUSPICIOUS	Yes ($f < n/3$)	Yes†, delayed	Yes†	Permitted
SOVEREIGN	Yes ($f < n/3$)	No	No (validator block stores only)	Locked
RECOVERING	Yes ($f < n/3$)	Pending recovery proof	Backfilling	Locked

Receipt tolerance. Let τ be the receipt tolerance: in ANCHORED and SUSPICIOUS, a valid block at height h carries a Bitcoin receipt and a DA receipt that attest settled and available heights $c_s, c_a \geq h - \tau$ (Section 3.8).

Theorem 1 (withdrawal safety). *Assume $f < n/3$, no Bitcoin reorganization deeper than $K_{DEEP_FINALITY}$, and an honest DA layer. Assume further that (M1) honest validators reject a block that contains a withdrawal while the state is* SOVEREIGN *or* RECOVERING*; (M2) each Bitcoin checkpoint commits the state root at its height; and (M3)* RECOVERING→ANCHORED *requires a verified recovery proof and a DA backfill certificate for the whole* SOVEREIGN *interval. If a withdrawal commits in block B_h, then P_{agree} holds, the state at h is* ANCHORED *or* SUSPICIOUS*, and there are heights $c_s, c_a \geq h - \tau$ such that $P_{settle}(c_s)$ and $P_{avail}(c_a)$ hold. Every height before $h - \tau$ is settled and available, including every* SOVEREIGN *interval that precedes B_h.*

Proof. P_{agree} follows from $f < n/3$. By (M1), the state at h is ANCHORED or SUSPICIOUS. In the transition function δ, SUSPICIOUS is entered only from ANCHORED, and SOVEREIGN is left only through RECOVERING; so every SOVEREIGN interval before h ended with a RECOVERING→ANCHORED transition, which by (M3) required a verified recovery proof and a DA backfill certificate. Hence the interval is valid, hash-linked, and available. In the ANCHORED and SUSPICIOUS blocks since then, every block carries receipts within τ. The Bitcoin receipt in B_h attests a checkpoint at depth at least $K_{\text{DEEP_FINALITY}}$ for some $c_s \geq h - \tau$; by (M2) it commits the state root at c_s, so rewriting the prefix up to c_s requires a deeper reorganization, which is excluded. The DA receipt in B_h attests data availability up to some $c_a \geq h - \tau$, and the DA layer is honest. □

Theorem 1 states the exit condition with its exposure window: the τ most recent blocks are neither settled nor available when the withdrawal commits. Liveness comes from relaxing the receipt checks in SOVEREIGN; user safety comes from tying withdrawals to the settled and available prefix. The prototype implements the three assumptions as follows. (M1) is enforced in the validity predicate and the ante handler. (M2) holds because each checkpoint payload carries the CometBFT block hash at its height. (M3) holds in the DA half: the exit edge requires the backfill condition, which the FSM safety run checks as an action property. The proof half of (M3) holds only when the heuristic recovery path is disabled (Section 7.3). The theorem is a pen-and-paper argument over these assumptions, not a mechanized statement about the implementation.

3.4. Health Certificates

The health certificate turns asynchronous, possibly inconsistent local observations into a single value that every validator can recompute.

Observation. At height $h-1$, validator i with voting power w_i produces an observation $o_i = (m_i, \sigma_i)$, where m_i is its local metric vector – Bitcoin gap, anchored and current Bitcoin heights, anchor re-verification result, DA gap and verified height, DA sampling and attestation flags, and the six P2P-quality bounds – and σ_i is i's signature over CometBFT's vote-extension sign bytes $(\mathit{chainID}, h-1, r, m_i)$. The extension is produced in `ExtendVote`, the only ABCI++ hook permitted to read live sensors.

Certificate. The proposer of height h receives the extended commit of height $h-1$ and places a certificate $C^h = (h-1, \{o_i\}_{i \in S})$ as the first transaction of its proposal. C^h is *well-formed* iff (a) every σ_i verifies against a genesis validator key for height $h-1$; (b) each validator contributes at most one observation; and (c) $\sum_{i \in S} w_i \geq 2f+1$.

Aggregation. $\mathsf{Agg}(C^h)$ maps each numeric metric to the w-weighted median of $\{m_i\}_{i \in S}$ and each Boolean metric to its w-weighted majority. Agg is a pure function: every validator that receives C^h computes the same value without contacting any peripheral network.

Lemma 1 (honest-bounded aggregate). *If C^h is well-formed, then for every numeric metric, $\min_{i \in S \cap H} m_i \leq \mathsf{Agg}(C^h) \leq \max_{i \in S \cap H} m_i$, where H is the set of honest validators; and every Boolean metric on which all honest members of S agree takes that value. Proof sketch.* Let $W = \sum_{i \in S} w_i \geq 2f+1$. Byzantine weight in S is at most $f < W/2$, so honest weight in S strictly exceeds $W/2$. A weighted median cannot lie outside an interval holding more than half the weight, and a weighted majority cannot overturn a value holding more than half the weight. □

Lemma 1 does not require honest observations to agree; it bounds what the adversary can do when they disagree. Each adversarial case reduces to it:

- *Stale observation.* A signature binds the height; an observation replayed from an earlier height fails (a) and is discarded.
- *Conflicting observations.* Two different signed observations by one validator count once by (b); Byzantine weight is bounded by f either way.
- *False observations.* Up to f colluding validators reporting extreme values cannot move the aggregate outside the honest range (Lemma 1).
- *Proposer omission.* The proposer chooses S, and may omit honest observations. If S still meets (c), Lemma 1 still holds. The proposer's residual influence is choosing *which* honest

value in the honest range is selected when honest validators genuinely disagree. If omission drops S below quorum, the certificate is not well-formed and honest validators reject the proposal. The only exception is the first height after vote extensions are enabled, whose predecessor carries no extensions; there the certificate is replaced by a maximally degraded metric vector, the conservative choice. [PENDING: re-run E8 live with the omission attack (B3).]

- *Missing observations.* CometBFT's extended commit for $h-1$ always contains extensions from at least $2f+1$ voting power, so an honest proposer can always build a well-formed certificate; missing observations from up to f validators do not block progress.
- *Divergent honest views.* When honest views straddle a threshold, different proposers may select certificates on different sides of it. Each resulting transition is legal and deterministic given its certificate; the FSM's hysteresis absorbs short alternations (Section 3.7).

Freshness. The certificate committed at height h describes observations made at height $h-1$, one block of lag. Staleness beyond that is impossible for a well-formed certificate, because signatures bind the height.

3.5. Finite State Machine

The FSM has four states (Table 4). ANCHORED is normal operation: blocks are valid only with a Bitcoin receipt and a DA receipt within tolerance, and withdrawals are permitted. SUSPICIOUS is an early-warning state reached after a sustained warning; withdrawals remain permitted, but admission of forced transactions is rate-limited by a bounded queue. SOVEREIGN is the fallback: the Bitcoin and DA receipt checks are relaxed for unhealthy layers, blocks continue under the validator quorum alone, and the circuit breaker locks all cross-chain withdrawals. RECOVERING is entered once the certificate reports healthy peripherals; withdrawals stay locked until a recovery proof over the SOVEREIGN interval is verified and the recovery hysteresis is satisfied (Section 3.9).

Table 4: FSM States

State	Validity requires	Withdrawals	Forced-tx admission	BFT finality	BTC settlement
ANCHORED	BTC + DA receipts	Permitted	Full	∼1–2 s	Yes (K_{DEEP} conf.)
SUSPICIOUS	BTC + DA receipts	Permitted	Rate-limited	∼1–2 s	Yes, delayed
SOVEREIGN	Quorum only	Locked	Full	∼1–2 s	No
RECOVERING	Quorum + proof	Locked	Full	∼1–2 s	Pending proof

BFT finality is CometBFT's deterministic finality and is unaffected by the FSM state; the values are the measured block interval on our testbed. Bitcoin settlement is probabilistic and follows Bitcoin's own confirmation time (K_{DEEP} blocks, ∼10 min per block on mainnet); it is not measured in this paper.

Fig. 3 shows the transition function δ. A proposal carries a target state s^h; honest validators prevote for it only if $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$. Since Agg and δ are deterministic and s^{h-1} is committed, every honest validator reaches the same verdict on a given proposal, and the committed state inherits CometBFT's agreement guarantee.

3.6. Network Sensors

Sensors are the only non-deterministic component; their outputs enter consensus only through the certificate.

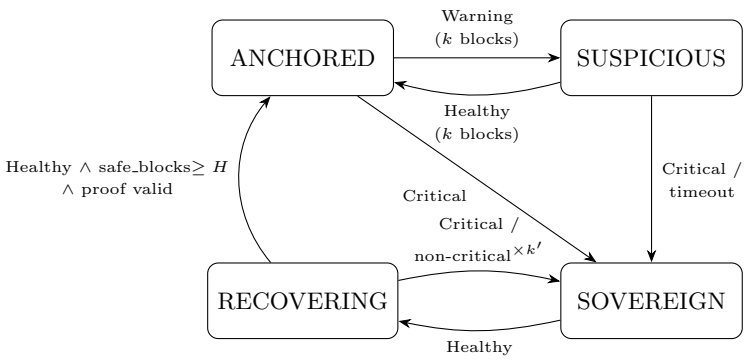


Figure 3: FSM transition function δ. Edges into ANCHORED and the non-critical edge out of RECOVERING are gated by hysteresis counters (k, H, and k', where k' doubles after each failed recovery up to a cap). Critical conditions bypass hysteresis.

Bitcoin finality gap. $\Delta H_{\text{BTC}} = H_{\text{current}} - \min(H_{\text{submitted}}, H_{\text{anchored}})$ measures how far the latest checkpoint lags Bitcoin's tip; it grows under congestion or checkpoint censorship. Because a delay metric cannot detect a checkpoint that was reorganized away, each validator also re-verifies, against its own Bitcoin full node, that the checkpoint transaction is still included at depth $\geq K_{\text{DEEP_FINALITY}}$. A failed re-verification is a critical condition.

DA gap. $\Delta H_{\text{DA}} = H_{\text{local}} - H_{\text{verified}}$ measures the lag between the chain head and the highest height whose DA submission the validator has confirmed as retrievable from its own Celestia bridge node. Section 5.9 describes what the prototype publishes and how this differs from full sampling.

P2P health. Peer count alone cannot distinguish a long-lived honest peer from a fresh Sybil, and eclipse attacks combine vectors no single metric captures [61, 62]. The profiler combines six bounds – subnet diversity, active anchor peers, clean peers, churn rate, mean peer tenure, and latency – applied to the Engram P2P interface, the Bitcoin client, and the Celestia client. These bounds characterize peripheral and partial P2P degradation; a partition of the core consensus network below $2f+1$ is outside what any FSM can handle (Section 1).

3.7. Transitions and Hysteresis

A *warning* (suspicious Bitcoin gap, unhealthy DA, or unhealthy P2P quality) demotes ANCHORED to SUSPICIOUS only after k consecutive unhealthy blocks. A *critical* condition (excessive Bitcoin gap, failed anchor re-verification, complete anchor-peer isolation, or a SUSPICIOUS timeout) moves the chain to SOVEREIGN immediately. Recovery is deliberately slower than degradation: SUSPICIOUS→ANCHORED requires k consecutive healthy blocks and RECOVERING→ANCHORED requires H consecutive healthy blocks plus a verified recovery proof. To resist an adversary who repeatedly toggles a peripheral to force failed recoveries, the non-critical demotion threshold k' on RECOVERING doubles after each RECOVERING→SOVEREIGN regression, up to a cap. Section 6.3 evaluates all three hysteresis-gated edges.

3.8. Extended Proposal and Validity Predicate

A proposal at height h carries, in $Txs[0]$, the certificate C^h, the target state s^h, a health flag, a DA receipt (last DA-verified height and attestation flag), a Bitcoin receipt (checkpoint height and block

hash), and, when applicable, a recovery-proof reference. `ProcessProposal` accepts it iff: (1) C^h is well-formed; (2) $s^h = \delta(s^{h-1}, \mathsf{Agg}(C^h))$ and the health flag matches $\mathsf{Agg}(C^h)$; (3) in ANCHORED and RECOVERING, and for any layer the certificate reports healthy, the DA and Bitcoin receipts are consistent with $\mathsf{Agg}(C^h)$ within a round-adaptive tolerance; (4) no withdrawal is included while s^h is SOVEREIGN or RECOVERING; and (5) a transition RECOVERING→ANCHORED is backed by a verified recovery proof. `ProcessProposal` never reads a live sensor. The FSM state is written to replicated state only by `PreBlocker` after commit, from the certificate already agreed upon.

Condition (3) is exactly what a *hard-gated* modular chain enforces unconditionally. Engram's only liveness-relevant change is to make (3) conditional on the certified state: when the quorum certifies a peripheral as unhealthy, the corresponding receipt is no longer required, and the circuit breaker (4) removes the operations whose safety depended on it.

3.9. Recovery Proof and Re-anchoring

Statement. On leaving SOVEREIGN, a prover builds a Noir circuit proof (Barretenberg UltraHonk backend [69, 75]) that, for a contiguous range of $n \leq N_{\max} = 256$ headers starting from the last anchored root rt_{last}: (a) each header links to its predecessor through a Poseidon2 header hash; (b) every FSM transition in the range is legal under δ and withdrawals were locked whenever the state was SOVEREIGN or RECOVERING; (c) the circuit breaker was respected; (d) the final header commits to the claimed state root rt_{new}; and (e) the boundary header carries valid ed25519 signatures from at least two-thirds of the voting power of the validator set, whose hash the header commits to (the compiled circuit fixes the set size at seven). Unused slots are padding, so proving time, verification time, and proof size are constant in n (Appendix B).

Intended verifier. A CometBFT quorum certificate shows that a quorum signed a block. The recovery proof additionally shows that what the quorum signed obeys the FSM transition rules and the withdrawal lock, including when the quorum is dishonest. This extra guarantee matters only to a verifier that does not trust the quorum: a bridge or light client on another chain, a node syncing from genesis, or a Bitcoin observer when the checkpoint commits the state root (Section 3.3). Validators already trust their own quorum, so for them the proof is a consistency check. The target verifier of this paper is an external light client that accepts a re-anchoring checkpoint only with a valid proof; the prototype does not include such a client.

Longer intervals. An interval longer than 256 headers is covered by a sequence of proofs: each accepted proof advances the on-chain rt_{last} to its rt_{new}, and the next proof must start from that root. This costs $\lceil L/256 \rceil$ proofs for L blocks. An aggregator circuit can fold up to four chunk proofs into one proof of the same size, which makes verification cost independent of L up to 1024 headers at the price of one extra proof (Appendix C).

Who verifies what. The recovery proof is submitted as a transaction and verified by *every Engram validator* during block execution, using an embedded, deterministic WebAssembly build of the UltraHonk verifier; its public inputs are bound to the committed rt_{last}, header count, and boundary height, and a verified height is latched so a proof cannot be replayed. Bitcoin does *not* verify the proof – Bitcoin Script cannot express UltraHonk verification. Bitcoin's role is the same as in ANCHORED mode: after recovery, validators resume writing `OP_RETURN` checkpoints that timestamp Engram heights, and the Bitcoin gap sensor requires these to reach depth $K_{\text{DEEP_FINALITY}}$. A checkpoint is a 44-byte payload: a 4-byte tag, the Engram height, and the CometBFT block hash at that height. Anchor re-verification compares this hash with the local block hash, so a Bitcoin observer can bind a height to a Bitcoin-timestamped root. The proof's rt_{new} is a Poseidon2

accumulator, not the block hash, and is not itself written to Bitcoin.

Resulting guarantee. The recovery proof guarantees, to anyone who verifies it, that the SOVEREIGN interval is a single hash-linked chain signed at its boundary by a validator quorum, obeyed the FSM rules, and released no withdrawals. It does not make the interval's *ordering* secure against a Byzantine supermajority: during SOVEREIGN, safety rests on the f-bounded validator quorum alone, which is precisely why withdrawals are locked. Re-anchoring restores Bitcoin timestamping from the recovery point forward.

Data availability of the Sovereign interval. A recovery proof shows the interval is *valid*, not that its data is *available*. We therefore strengthen the exit condition: RECOVERING→ANCHORED requires (1) a verified recovery proof and (2) a DA backfill certificate – a statement, backed by the quorum of signed observations in the health certificate, that the block data for every height of the SOVEREIGN interval has been published to the DA layer and is retrievable. During SOVEREIGN, each validator keeps the interval's blocks in its own store (they are ordinary committed CometBFT blocks) and queues them for publication; on recovery the queue is drained in height order. Each validator reports a contiguous height: the highest height such that every height up to it has been confirmed retrievable. The certificate takes the weighted median of these reports, and the exit edge requires the median to reach the height at which RECOVERING was entered, less one. The DA gap sensor keeps measuring head liveness, so the DA layer stays healthy while the backfill drains. The specification abstracts the condition as a Boolean, `da_backfill_valid`. The prototype implements the queue and the gate; Section 5.9 describes the remaining live validation.

4. Formal Specification and Bounded Model Checking

4.1. Layered Specification

Model-checking consensus, FSM, and sensors together is intractable at useful scale. We therefore specify the protocol in TLA+ [70] as four layers – an abstract consensus object, a Tendermint-style consensus layer, the FSM layer, and a server layer composing them – and check each layer with TLC and, for bounded safety searches, Apalache [42]. The target property is: under partial synchrony and an f-bounded adversary, no two honest validators commit conflicting blocks or conflicting FSM states, and any such conflict implies double-signing evidence.

What is checked, and what is not. Two levels of evidence must be kept apart.

- *Per-layer bounded model checking (complete).* Each layer's own safety invariants and liveness properties are checked in isolation at its own bounded configuration (Table 5), together with the ablation study (Table 6).
- *Cross-layer refinement (open).* The obligation that the composed server layer refines the abstract consensus object – which is what would transfer the target property to the composed system – is specified at three scales (C1: N=4, f=1; C2: deeper rounds; C3: N=7, f=2) but has not converged at any of them. The composed system is therefore *not* formally verified end-to-end; its whole-system evidence in this paper is empirical (Section 6).

To make C1 tractable we split it by which actions may fire and check each part with TLC under symmetry reduction over the three honest validators. With the consensus core frozen, the environment part (sensor updates, clock ticks, DA backfill) completed: 17,500 distinct states, depth 10, 10 m 35 s, no violation. With the environment frozen, the consensus-core part did not complete: it was stopped after 1 h 41 m at 19,848,141 distinct states (depth 15, 13,542,792 states still queued)

with no violation. Neither part covers the interleaving of environment and consensus steps, and no composition argument is established, so C1 is not claimed.

Running these parts exposed a defect in the server layer: the vote-extension action left the Tendermint variables unconstrained, so no earlier server-layer run could have passed its first such step. The fix, two guards that stop the server from certifying a timeout for a round an honest validator has already left, and one added abstract action for a round ended by a quorum of nil precommits are changes to the checked model. The last weakens the target specification. An inductive check of every server action with Apalache would avoid the composition gap; it has not been attempted.

Safety versus liveness. The linking invariant *QuorumOverlap* – any two valid quorums share at least one honest validator – is a *safety* premise: together with honest validators' refusal to sign conflicting votes, it prevents two conflicting decisions. It does not establish liveness. Following LiDO [47], liveness is argued separately, and in our specification it rests on explicit fairness assumptions that the model checker takes as hypotheses:

- *Partial synchrony:* GST is reached infinitely often (`[]<>GSTReached`), and after GST messages between honest validators are delivered (weak fairness on each node's `Push` and `Pull`).
- *Timeout progress:* weak fairness on round-timeout expiry and on the $f+1$ round-skip (fast-forward) action.
- *Proposer progress:* weak fairness on each honest node's invocation step, so every honest proposer whose round is reached after GST eventually proposes.
- *FSM progress:* weak fairness on the FSM transition and on environment updates, so a persistently healthy (or unhealthy) certificate eventually drives the corresponding transition.

Under these hypotheses TLC checks the FSM liveness properties directly. A TLC ablation that removes the $f+1$ fast-forward, which would show the timeout-progress assumption is necessary, has not completed: its best run reached depth 12 (69,408,583 states generated, 828,673 distinct) with no violation. The necessity of that assumption is therefore not established here.

Health certificates in the model. The server layer models the certificate explicitly. Each validator's observation holds three Booleans (healthy, warning, critical) and the Bitcoin and DA gaps clipped to 0..3, written to `sensor_view` by `ServerExtendVote`. A Byzantine validator may sign any observation; an honest validator with an eclipsed P2P layer may report a degraded one, never a healthier one. The proposer may choose any member set of at least $2f+1$. The aggregate (`AggregateHealthCertificate`) is a per-field majority for the Booleans and a weighted median for the gaps, and `IsValidProposal` rejects a certificate below $2f+1$.

Lemma 1 is checked as `AggWithinHonestRange` with Apalache, with the observation vector left symbolic so that one check covers every vector. For $N=4$ and $f=1$ it reported no violation (22 s). With two Byzantine validators in the same network it reports a violation (25 s), so the bound on Byzantine members is necessary. Signatures, height binding, and the individual P2P counters are not modelled. The check covers the aggregate as a function; the certificate's role inside `IsValidProposal` is covered only by the open refinement obligation.

4.2. Safety Invariants and Attack Resilience Lemmas

Six state invariants are specified and model-checked.

- *Circuit Breaker Isolation:* withdrawals are locked if and only if the state is SOVEREIGN or RECOVERING.
- *Hysteresis Integrity:* RECOVERING→ANCHORED requires the full hysteresis wait and a valid recovery proof.
- *Suspicious-Exit Hysteresis Integrity:* the corresponding guard on SUSPICIOUS→ANCHORED.
- *Strict FSM Transition Safety:* only transitions drawn in Fig. 3 occur.
- *Monotonicity Safety:* heights and time never decrease.
- *Sovereign-Data Publication:* every RECOVERING→ANCHORED transition occurs with the DA backfill condition true. This is an action property, checked in the FSM safety run; the Server-layer drivers have not been re-run with it.

Five attack-specific properties are enforced by the validity predicate: resistance to data withholding; resistance to forged or reorganized anchors through anchor re-verification; a deterministic bound on Byzantine timeout flooding; resistance to eclipse-driven recovery through the six-bound P2P gate; and accountability, i.e. any safety violation implies double-signing evidence. Accountability is inherited from CometBFT's equivocation evidence pool rather than proved anew. All of these properties are conditional on the Bitcoin reorg-depth bound of Section 3.2.

4.3. Verification Stress Test and Ablation Study

Table 5: Per-Layer, Non-Refinement Verification Results (TLC)

Layer / property	States gen.	Distinct	Depth	Time	Workers	Viol.
Tendermint (safety)	3,772,809	537,536	25	1 m 01 s	10	0
FSM (safety)	1,371,377,696	4,542,271	23	10 m 19 s	10	0
FSM (liveness)	17,986,105	31,105	23	3 m 45 s	10	0

Each row is one completed TLC run of that layer's driver. Depth is the number of breadth-first levels at which the search terminated with every state satisfying the state constraint explored, so it is the diameter of the constrained state graph and not a cap. Hardware: Apple M1 Pro, 10 cores, 32 GB RAM.

Table 7 lists the bounds of each run.

Table 5 reports the per-layer results, each obtained by running that layer's own model-checking driver standalone at its checked-in configuration (Table 7). All three runs found zero violations in their bounded state spaces. These results cover the Tendermint-style and FSM layers; the abstract and server layers are exercised only through the open refinement obligation above.

The ablation study spans two layers: the hysteresis and P2P health gate belong to the FSM layer, whereas the circuit breaker and DA consistency are checked in the Tendermint-layer driver. Each result therefore holds within its own layer's bounded configuration.

Table 6 reports the ablation study. Each row removes one mechanism and re-runs verification against the sanity property that mechanism protects. Apalache's bounded SMT search found each counterexample in 24–117 s at depth 2–6, far faster than TLC's breadth-first search for shallow counterexamples. Each ablation reproduced the violation its mechanism is designed to prevent: without the circuit breaker a withdrawal leaks; without hysteresis the chain re-anchors prematurely; without the P2P gate an eclipsed validator forces an illegal recovery; and without DA consistency a

Table 6: Ablation Study: Removing One Mechanism at a Time (Apalache)

Removed	Targeted threat	Depth	Time	Violated property
Circuit breaker	Withdrawal leakage	2	66.7 s	`Sanity_NeverAttempt WithdrawalLeakage`
Hysteresis	Premature re-anchoring	6	117.0 s	`Sanity_NoIllegal HysteresisExit`
P2P health gate	False recovery under eclipse	3	24.2 s	`Sanity_NoIllegal P2PRecovery`
DA consistency	Data withholding	2	63.3 s	`Sanity_Never ProposeWithheldData`

withholding proposal is accepted. The ablations show what each mechanism prevents. Single-factor ablation does not test joint removal, so it does not establish that the set of mechanisms is sufficient.

Table 7: TLC Run Configurations

Run	Constants and bounds
Tendermint (safety)	N=4, f=1 (three honest, one Byzantine node); MaxRound 2; MaxBTCHeight 10; MaxEngramHeight 10; MaxTimestamp 2; symmetry reduction over the three honest nodes
FSM (safety, liveness)	SuspiciousThreshold 2; SovereignThreshold 4; DAThreshold 2; RecoveryHysteresisWait 2; SuspiciousHysteresisWait 2; MaxSuspiciousTime 2; DownHysteresisThreshold 2; MaxDownHysteresisThreshold 8; MinPeers 3; MinSubnetDiversity 2; MinAnchorPeers 1; MaxChurnRate 10; MinAvgTenure 100; MaxPeerLatency 50; the Bitcoin and Engram heights stay below 10 (safety run); no symmetry reduction

5. Experimental Setup

5.1. Testbed

The testbed runs the reference implementation – a Cosmos SDK application on a forked CometBFT core – as seven validators with equal voting power (n=7, f=2), each in its own container. Validator gossip uses a dedicated full mesh of pairwise links, separate from the network that carries Bitcoin, Celestia, and RPC traffic; this separation lets us degrade peripheral connectivity without partitioning the core consensus network. The peripheral layers are real software, not simulators: a two-node Bitcoin regtest network mined continuously at roughly one block per 20 s, and a Celestia network with seven bridge nodes, one per validator, so that each validator forms its own DA view. A prover process (Noir/Barretenberg) watches for a Sovereign backlog and submits recovery proofs. The eclipse and Sybil rows (A1 and A2) of Appendix A predate the move to seven validators and ran on a four-validator cluster (f=1). [PENDING: re-run A1/A2 at n=7, and run the f=2 collusion suite of Section 6.1.]

5.2. Environments and Software Versions

All results come from one environment, Testbed A: the seven-validator Docker deployment above on a single host. A second environment, Testbed B, with one validator on each of seven cloud virtual machines, is planned for the experiments that depend on real wide-area conditions. Every table and figure in Section 6 is a Testbed A measurement unless its caption says otherwise, and results from

the two environments are never pooled into one number. Table 8 lists the configuration.

Table 8: Testbed Configuration and Software Versions

Component	Testbed A	Testbed B
Host	Apple M1 Pro (10 cores: 8P + 2E), 32 GB RAM, 1 TB SSD; macOS 27.0	[PENDING:]
Container runtime	Docker 29.0.1; VM allocation 10 CPUs, 23.4 GiB RAM	[PENDING:]
Validators (7)	No CPU or memory limit per container	[PENDING:]
Consensus core	CometBFT v0.39.3 module path, replaced by the fork `engram-consensus-core` at commit `493a20121`	Same commit
Application	Cosmos SDK v0.54.3; this repository at [PENDING: results commit]	Same commit
Bitcoin	Bitcoin Core v25.0, regtest, two nodes (0.3 CPU, 512 MB each)	[PENDING:]
Celestia	celestia-app v1.11.0 (0.5 CPU, 1 GB); seven celestia-node v0.14.1 bridges (0.3 CPU, 512 MB each)	[PENDING:]
Prover	nargo 1.0.0-beta.22, `bb` 5.0.0-nightly.20260522 (6 CPU, 8 GB limit)	[PENDING:]
Fault injection	Pumba (image tag `latest`) driving kernel `tc netem`	Real network
Formal tools	TLC 2026.04.18 (rev. c45c125); Apalache v0.58.3 on JDK 17	—

CPU and memory figures are container limits. The fork differs from upstream CometBFT in four ways: an $f+1$ timeout round-skip, per-peer round-trip-time measurement and a peer health monitor that feed the P2P sensor, per-peer rate limiting and bounded round acceptance for timeout messages, and the proposal round passed to the application through `PrepareProposal` and `ProcessProposal`.

5.3. Fault Injection

Network-level faults use kernel traffic control (`tc netem` via Pumba): uniform delay and jitter, packet loss, single-node isolation, and a heterogeneous per-validator wide-area profile approximating seven regions. Appendix E lists the parameters of every profile. The *WAN latency baseline* is the per-validator delay and jitter profile of Table 20, held for the whole run unless a step in Table 9 suspends it.

Application-level faults reproduce mechanisms that delay alone cannot, and each simulates the consequence of a failure rather than its cause.

- *Bitcoin congestion* suspends the submission of new checkpoints (a pending one still confirms), so the finality gap grows directly. It does not congest the Bitcoin mempool.
- *DA outage* stops one or all seven Celestia bridge containers. It does not emulate a data-availability-sampling failure.
- *Byzantine behavior* is injected through attacker processes or through a validator configured to forge proposal fields, omit transactions, double-sign, or flood timeout messages.

Every fault targets a peripheral layer or a minority of validators; none partitions the core consensus network below $2f+1$, consistent with the scope defined in Section 1.

5.4. Fault Schedules

Table 9 gives the onset, mechanism, and duration of each fault in the two scripted runs of Section 6.2. Times are relative to the start of each run, durations are the length of the polling window the script

holds the fault for, and each of S2–S5 is followed by a wait of up to 90 s for the validators to return to ANCHORED before the next scenario starts.

Table 9: Fault Schedules of the Scripted Runs

Step	Fault and mechanism	Duration
Composite run (standing WAN latency baseline)		
C1	Baseline, no fault	15 s
C2	Bitcoin fault: 500 ± 100 ms RPC delay on one `bitcoind` (profile length 2 min)	60 s
C3	All seven Celestia bridges stopped, added to C2	60 s
C4	Three cycles of 5% packet loss on validators 1 and 2 (WAN baseline suspended)	$3 \times$ (20 s on, 10 s off)
C5	Hold; bridges still stopped	30 s
C6	Heal: Bitcoin profile cleaned up, bridges restarted	30 s
C7	Wait for ANCHORED (limit 600 s)	until recovery
Seven-scenario sweep		
S1	Healthy baseline	15 s
S2	Checkpoint submission suspended on all validators	180 s
S3	All seven Celestia bridges stopped	60 s
S4	300 ± 50 ms delay on the six validator links of validator 1	120 s
S5	100% packet loss on validator 1	180 s
S6	One `bitcoind` and all seven bridges stopped	90 s
S7	Faults removed; wait for ANCHORED (limit 600 s)	until recovery

5.5. FSM Parameters

FSM parameters are fixed in the genesis file, so every validator holds identical values. All live experiments use the defaults of Table 10 unless a section states an override. Sensors are sampled once per block, when a validator signs its vote extension; the measurement scripts poll each validator's RPC every 3 s (2 s during the packet-loss cycles of C4). Blocks arrive about once per second.

Table 10: FSM Parameters Used in the Live Experiments

Parameter	Value	Unit and role
`SuspiciousThreshold`	5	Bitcoin gap, blocks
`SovereignThreshold`	8	Bitcoin gap, blocks
`DAThreshold`	30	DA gap, blocks
`KDeepFinality`	2	Reorg depth, Bitcoin blocks
`MinPeers`	3	Clean peers
`MinSubnetDiversity`	2	Distinct subnets
`MinAnchorPeers`	2	Active anchor peers
`MaxChurnRate`	5	Peer churn events
`MinAvgTenure`	300	Mean peer tenure, s
`MaxPeerLatency`	200	Round-trip time, ms
`RecoveryHysteresisWait`	2	RECOVERING→ANCHORED, blocks
`DownHysteresisThreshold`	2	ANCHORED→SUSPICIOUS, blocks
`MaxDownHysteresisThreshold`	8	Cap on the backed-off threshold
`SuspiciousHysteresisWait`	2	SUSPICIOUS→ANCHORED, blocks
`MaxSuspiciousTime`	24	SUSPICIOUS escalation bound, blocks

Overrides: the hysteresis sweeps of Section 6.3 vary one hysteresis parameter at a time; the Monte Carlo study of Section 6.1 uses the P2P thresholds of Section 5.7.

5.6. Metrics

Let a run record the FSM state s_b of every block b in a window W.

- *Anchored uptime* $= |\{b \in W : s_b = \text{ANCHORED}\}|/|W|$. In-process, W is the whole recorded timeline: for the noisy environments, the 100 noisy blocks plus the setup blocks that precede them (two for the RECOVERING→ANCHORED sweep, one for the others). The no-noise control of that sweep ends at the first ANCHORED block, so its window is only the setup blocks plus the wait, and its uptime falls as the wait grows. Live, W is the sequence of RPC samples of one validator over a 300 s window.
- *Flapping count* is the number of transitions that immediately reverse the preceding transition, for example ANCHORED→SUSPICIOUS followed by SUSPICIOUS→ANCHORED, over the same W. Live runs detect reversals between consecutive samples, so a reversal faster than the polling interval is not seen.
- *Absorption rate* is $a/(a+d)$, where a is the number of disturbed blocks that begin in ANCHORED and leave the state unchanged, and d is the number that move it to SUSPICIOUS. It is defined only on the ANCHORED→SUSPICIOUS and SUSPICIOUS→ANCHORED sweeps.
- *Blocks/s* $= (h_{\max} - h_{\min})/T$ over a measurement window of length T (60 s after an 8 s warm-up in Section 6.4). *Steady blocks/s* $= 1/\bar{\imath}$, where $\bar{\imath}$ is the mean block interval. *Gap fraction* $= 1 - n\bar{\imath}/T$, where n is the number of observed intervals: the share of the window not accounted for by intervals at the steady pace, that is, time spent in heights that took longer than one interval.

5.7. Synthetic Workloads

Noise profile (in-process). The hysteresis sweeps drive the real state machine with sensor readings generated per block. Each block is disturbed independently with probability 0.2 and healthy otherwise, for 100 blocks, from a fixed pseudo-random seed (7 for the RECOVERING→ANCHORED sweep, 11 for the ANCHORED→SUSPICIOUS sweep), so every threshold value sees the same noise sequence. A disturbance sets sensors to a failing reading for one block. Critical disturbances set the Bitcoin gap to `SovereignThreshold`. Warning-level disturbances set the Bitcoin gap to `SuspiciousThreshold`, mark DA unavailable, or give the P2P sensor a snapshot with no clean peers, diversity, or tenure and only the minimum anchor count. The combined environments apply several at once.

Monte Carlo P2P study. A *snapshot* is one synthetic tuple of the six P2P sensor fields (subnet diversity, active anchors, clean peers, churn rate, mean tenure, latency). A benign snapshot draws each field inside its healthy range with a uniform random margin. An attack of *intensity* $x \in [0, 1]$ perturbs the fields it targets in proportion to x (for example, the relay attack adds x (`MaxPeerLatency`+150) ms of latency and leaves the other fields benign); $x = 0$ is benign and $x = 1$ is the full attack. Detectors are the six-bound profiler and the peer-count-only rule CleanPeers $\geq$ `MinPeers`. The study uses production-scale P2P thresholds: 8 subnets, 4 anchors, 20 clean peers, churn 5, tenure 300 s, latency 200 ms. Four measurements per cell follow, all from fixed seeds.

- *False-positive rate:* the share of 2,000 benign snapshots flagged unhealthy.
- *False-negative rate:* the share of 2,000 full-intensity snapshots flagged healthy.

- *Detection delay:* over 500 ramps, the number of snapshots until the first flag, with intensity rising $0.1, 0.2, \ldots, 1.0$ over 10 snapshots. The mean is taken over ramps that were detected.
- *Evasion threshold:* the largest intensity (on the 0–1 scale) at which a majority of 20 probes still read healthy, found by a 12-step binary search that queries the detector, so it bounds an adversary with oracle access.

The ramp-shape study uses 300 ramps per shape and the three shapes x (linear), $\sqrt{x}$ (front-loaded), and x^2 (slow). The upper 95% confidence bound on a rate observed as 0 in n trials is about $3/n$ (rule of three): 0.15% for $n = 2{,}000$ and 1% for $n = 300$. [PENDING: report the standard deviation of detection delay per cell.]

5.8. Baselines

Hard-gated modular baseline (primary, RQ2). The same binary with the FSM pinned to ANCHORED: condition (3) of the validity predicate (Section 3.8) is enforced at every height, so a proposal is valid only if it carries a Bitcoin receipt and a DA receipt within tolerance. This is the behavior of a chain that makes settlement and DA inclusion part of block validity, and it isolates the effect of Engram's adaptive relaxation. [PENDING: implement the pinned-ANCHORED mode and run the RQ2 fault schedule against it.]

Vanilla CometBFT (control, RQ4). The same application with the health-aware ABCI++ hooks disabled. It does not depend on Bitcoin or Celestia and therefore never stalls when they fail; it serves as the cost reference for RQ4 and as a control showing that measured stalls or fallbacks are not artifacts of the harness.

Ablation baselines. A peer-count-only P2P detector isolates the contribution of the six-bound profiler (RQ1), and each hysteresis edge with its threshold set to its floor isolates the contribution of hysteresis (RQ3).

5.9. Sim-to-Real Gap

Four properties of the testbed limit how its results transfer to a production deployment.

- *Single physical host.* All Testbed A containers share one machine's CPU and memory, so resource contention can affect every validator at once; wide-area conditions are emulated with `tc netem` rather than real geographic distribution. Testbed B removes both limitations for the experiments repeated on it. [PENDING: repeat the composite-fault experiment on Testbed B (seven hosts in at least three regions, ≥ 3 trials) and report the measured inter-region round-trip times.]
- *Accelerated Bitcoin.* Regtest blocks arrive about 30× faster than on mainnet. Height-based thresholds are unaffected, but wall-clock durations scale accordingly.
- *DA payload.* The DA publisher submits the transactions of committed blocks in height-ordered batches of at most 1.5 MB and tracks the highest contiguous published height; the exit gate of Section 3.9 requires that height to cover the SOVEREIGN interval. The gate rests on a quorum-reported contiguous height, not on a cryptographic proof of retrievability. [PENDING: measure the time in RECOVERING against the interval length under a live DA outage and re-fetch every height of the interval from Celestia, comparing transaction hashes with the Engram blocks.]

- *Single-bridge trust.* Each validator trusts its own bridge node, which downloads and verifies full Celestia blocks, rather than sampling across independent peers; our DA-outage results measure bridge failures, not sampling-detected partial withholding.

6. Evaluation

Each research question is answered by the live experiments that bear on it most directly; the remaining attack scenarios, the prover-backend comparison, and the Bitcoin-reorg boundary test appear in Appendices A–D. Unless stated otherwise, all numbers are live measurements on Testbed A, the seven-validator testbed of Section 5 (Table 8); FSM parameters are those of Table 10, and metrics follow Section 5.6.

6.1. RQ1: Safety Under Byzantine Behavior

Single Byzantine proposer. The most direct attack on the extended consensus object is a proposer that claims an FSM state not implied by the certificate. One of the seven validators (n=7, 14.3% of the voting power) was configured to do so. Every honest validator recomputed $\delta(s^{h-1}, \mathsf{Agg}(C^h))$, rejected the forged claim in `ProcessProposal`, and the chain committed only honest proposals, with zero AppHash divergence. The same outcome held for the nine further single-actor attacks of Appendix A (eclipse, Sybil, data withholding, forged Bitcoin receipt, withdrawal in SOVEREIGN, censorship, a combined attack, double-signing, and timeout flooding).

Colluding Byzantine validators (f=2). The threat model allows f=2 colluding validators at n=7. We therefore define a collusion suite in which two validators act jointly: (B1) both report coordinated false health observations; (B2) one reports false observations while the other, as proposer, builds its certificate to favor them; (B3) the proposer omits honest observations, both down to exactly $2f$+1 voting power and below it; (B4) both double-sign at the same height; (B5) both attempt to include a withdrawal while in SOVEREIGN; and (B6) the proposer attaches a stale or forged Bitcoin or DA receipt. AppHash agreement is necessary but not sufficient evidence of safety, since honest validators could agree on an illegal transition. For every committed height we therefore also replay $\delta(s^{h-1}, \mathsf{Agg}(C^h))$ offline from the committed certificate and check that it equals the committed s^h, that the aggregate lies within the honest range of Lemma 1, and that no withdrawal commits while $s^h \in \{\text{SOVEREIGN}, \text{RECOVERING}\}$. [PENDING: run B1–B6 at n=7 and report per-case AppHash agreement, semantic-replay violations, and the honest-range check.]

Peripheral P2P quorum gate. A P2P detector is useful for safety only if its signal cannot be abused by a minority. We first compare the six-bound profiler (Section 3.6) with a peer-count-only detector in a Monte Carlo study (2,000 benign and 2,000 attack snapshots plus 500 ramps per cell; Section 5.7), then run the two most diagnostic attacks live.

Table 11 shows that the profiler detects every attack at every intensity an adaptive adversary could choose (evasion threshold ≤ 0.25), under three ramp shapes (linear, front-loaded, slow; 300/300 ramps detected per cell, 95% upper bound on the miss rate 1%) and regardless of concurrent Bitcoin or DA degradation. The peer-count-only detector detects none of the 300 ramps in any cell. The live runs show what happens after detection. In P3 (churn-based rotation: eight restart cycles on two of seven validators, 29% of voting power), the churn metric on both attacked validators crossed its threshold, yet no FSM transition was committed: two validators' signed observations cannot move a certified aggregate backed by $\geq 2f$+1 voting power. In P4 (a sustained 350±50 ms delay on all of one validator's P2P interfaces, observed by all seven validators), the chain moved

Table 11: P2P Detector Accuracy: Six-Bound Profiler vs. Peer-Count-Only (Synthetic; 2,000 benign and 2,000 attack snapshots, 500 ramps per cell)

	Six-bound profiler		Peer-count-only	
Attack	**Delay**	**Evasion**	**Delay**	**Evasion**
P1 Peer-slot exhaustion	2.23	0.17	never	1.00
P2 Multi-subnet Sybil	1.00	0.00	never	1.00
P3 Churn-based rotation	1.70	0.12	never	1.00
P4 Relay-node latency	2.71	0.25	never	1.00

Delay: mean snapshots to first detection over a 10-snapshot ramp ("never": no ramp was detected). Evasion: largest attack intensity on the 0–1 scale at which the detector still reads healthy. Both detectors have 0/2,000 false positives (95% upper bound 0.15%). The profiler has 0/2,000 false negatives at full intensity; the peer-count-only detector has 2,000/2,000, since each attack preserves the clean-peer count.

ANCHORED→SUSPICIOUS→SOVEREIGN at +39 s and +72 s, held SOVEREIGN for the 244-second attack, and returned to ANCHORED within 5 s of the delay clearing (Fig. 4).

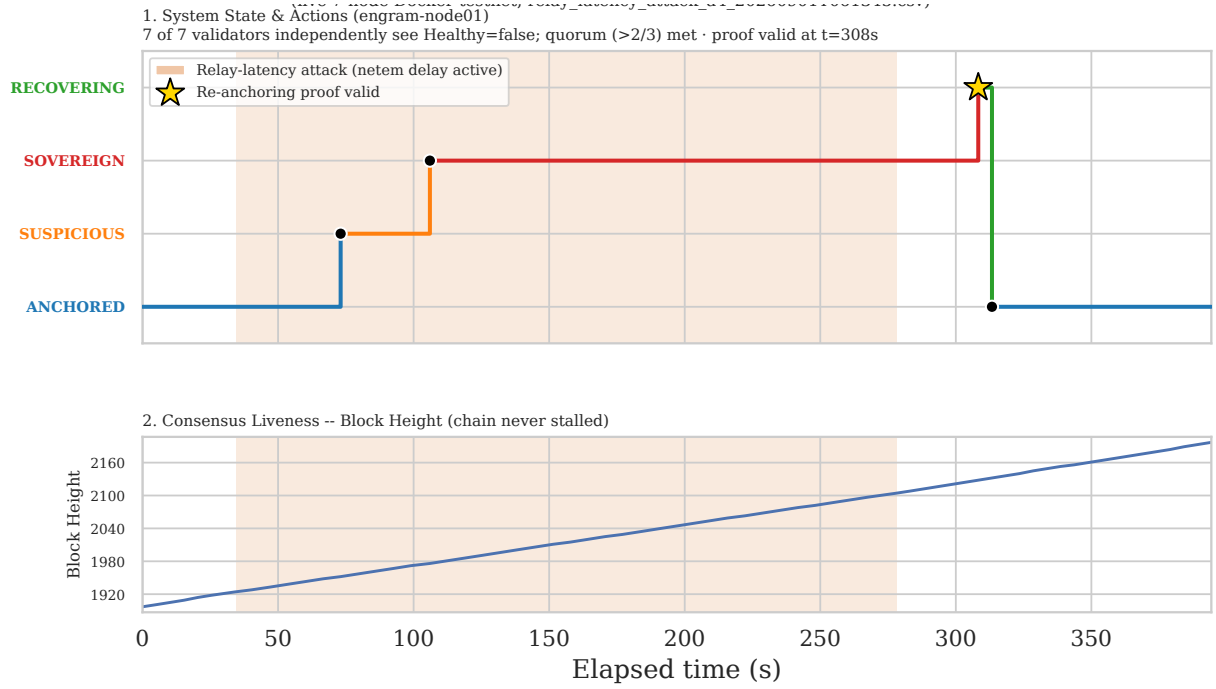


Figure 4: FSM cascade under a real, quorum-visible P2P relay-latency attack (P4): all 7 validators independently sense the degradation, the transition commits once quorum is met, and block production (bottom panel) never stalls through the cascade or the recovery.

P3 and P4 are two outcomes of one mechanism. The same kind of sensor-confirmed degradation is ignored when a minority observes it and drives a bounded cascade when all validators do, as Lemma 1 predicts: a minority cannot move the certified aggregate, whereas a degradation seen by every honest validator must move it.

6.2. RQ2: Liveness Under Peripheral Degradation

Engram versus a hard-gated modular chain. The hard-gated baseline (Section 5.8) enforces Bitcoin and DA receipts at every height. Under a DA outage, no proposal after the last retrievable height can carry a valid DA receipt; under checkpoint suspension, the Bitcoin receipt falls outside tolerance once the gap exceeds it. In both cases every proposal fails the validity predicate and the baseline commits no further blocks until the layer returns, while Engram moves to SOVEREIGN and continues. We run both systems through the same fault schedule and report blocks committed during each outage window and time to first commit after recovery. [PENDING: hard-gated vs. Engram: blocks committed per outage window (BTC suspension, DA outage, both), stall duration, and time to resume.]

Composite fault. Engram was then run through a continuous, scripted composite-fault sequence under a standing wide-area latency profile (Table 9): a Bitcoin fault, then a full DA outage (all seven bridges stopped), then three cycles of packet loss on the validator P2P network, healed in turn. The packet loss degrades peer quality without removing quorum connectivity. Fig. 5 shows the resulting timeline.

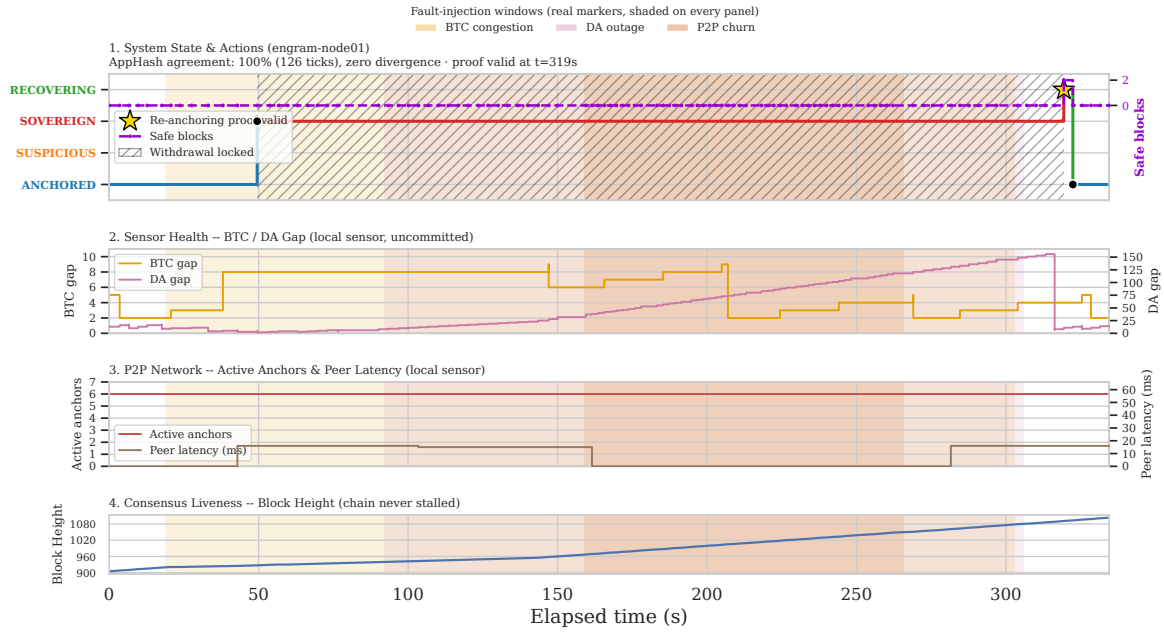


Figure 5: FSM state timeline, sensor readings, P2P network state, and block height under a layered composite fault sequence (Bitcoin congestion + DA outage + P2P churn, overlapping at their peak).

Over the 341-second run (742 samples, 126 polling ticks, one trial), the height advanced continuously from 905 to 1093 with no stalled height, and all seven validators agreed on the AppHash at every tick (zero divergent heights, zero RPC errors). The chain entered SOVEREIGN under the combined fault and, after the faults were healed, the recovery proof was verified at t=319 s, the chain entered RECOVERING at t=321 s and ANCHORED at t=324 s. Withdrawals stayed locked throughout SOVEREIGN and RECOVERING, as the circuit-breaker invariant requires. These timings come from a single trial. [PENDING: repeat the composite and seven-scenario runs ($\geq$ 5 trials) and report recovery time as mean $\pm$ 95% CI.]

Seven-scenario sweep. The composite trace does not isolate which failure mode drives which behavior. A second run therefore walks the chain through seven sequential scenarios in one 864-second run (one trial; Table 9): S1 healthy baseline, S2 Bitcoin congestion (checkpoint submission suspended), S3 DA outage (bridges stopped), S4 partial P2P eclipse, S5 anchor-peer isolation of one validator, S6 combined Bitcoin and DA failure, and S7 recovery. Each scenario starts from the state the previous one left; 153 FSM transitions were logged across the seven validators (Fig. 6).

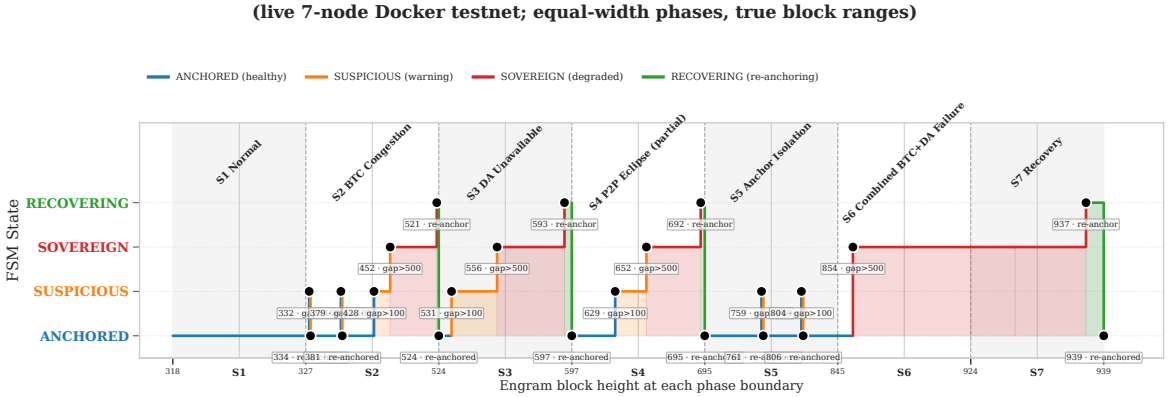


Figure 6: Continuous FSM-state timeline across seven sequential fault-injection scenarios (S1–S7), true block-height ranges, live 7-validator testbed. Each scenario begins where the previous scenario's recovery ended.

Each scenario produced the expected response. S2 and S3 each escalated through SUSPICIOUS to SOVEREIGN and recovered. S4 escalated to SOVEREIGN, whereas S5 produced only a transient SUSPICIOUS excursion absorbed by hysteresis: isolating one of seven validators from its anchor peers leaves the certified active-anchor-peer count above zero, so the critical condition never holds. S4 and S5 are therefore distinct failure classes rather than two intensities of one signal. S6 escalated directly to SOVEREIGN, and S7 returned to ANCHORED through a submitted and verified recovery proof. Two further observations stand out. First, S2 contains two brief SUSPICIOUS excursions (19–22 s and 77–80 s) that return to ANCHORED before the escalation at 141 s, showing hysteresis absorbing live rather than synthetic noise. Second, block production stayed close to baseline: block-interval means ranged 1.30–1.93 s across all seven scenarios (vs. 1.34 s at the S1 healthy baseline; all intervals in this paragraph are Engram's, and 1.34 s corresponds to the 0.743 blocks/s of Table 13), and the largest p95 intervals occurred in S4 (4.27 s) and S7 (3.10 s). Engram therefore keeps committing blocks both under overlapping faults and under each failure mode in isolation, and returns to ANCHORED in every case.

6.3. RQ3: Recovery and Flapping

Hysteresis should suppress oscillation ("flapping") without unduly delaying legitimate transitions. We swept the threshold on each of the three gated edges – RECOVERING→ANCHORED, ANCHORED→SUSPICIOUS, and SUSPICIOUS→ANCHORED – in-process under fixed-seed synthetic noise and live on the seven-validator cluster under real network noise (Fig. 7).

On the ANCHORED→SUSPICIOUS edge (Table 12), raising the down-hysteresis threshold from 1 to 4 collapses flapping from 31 events to 1 and raises anchored uptime from 61.4% to 98.0%, and the absorption rate saturates near 100% by threshold 4–6. The live spot-check changes the conclusion. Under wide-area jitter alone, with no injected fault, thresholds 1, 2, and 4 still flapped, and only

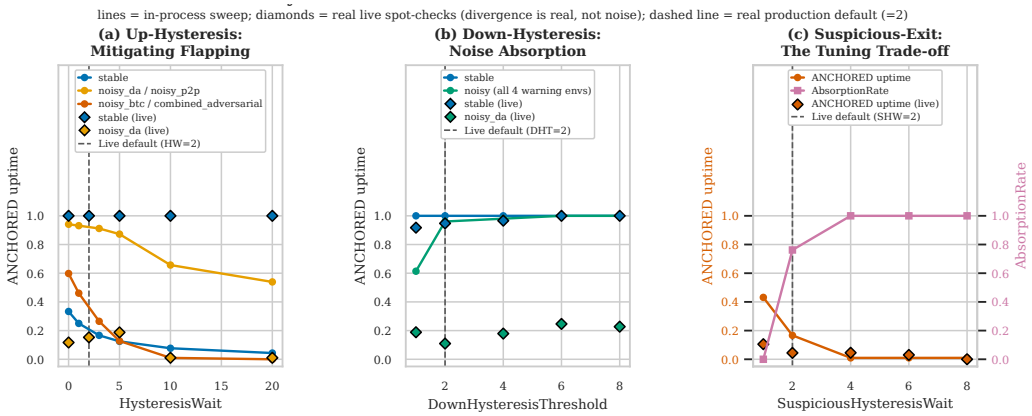


Figure 7: Hysteresis parameter sweep across all three absorb edges. Lines: in-process sweep under different noise profiles; diamonds: live spot-checks on the real 7-validator cluster (divergence from the in-process line is real network behavior, not measurement noise); dashed line: the protocol's live production default.

Table 12: Down-Hysteresis Sweep (ANCHORED→SUSPICIOUS), In-Process (101 blocks, noise probability 0.2, seed 11, one run per threshold)

Threshold	1	2	4	6	8
Flapping (count)	31	1	1	0	0
Anchored uptime (%)	61.39	96.04	98.02	100.00	100.00
Absorption rate (%)	0.00	94.74	95.24	100.00	100.00

thresholds ≥ 6 were clean (0 flaps, 100% uptime). The default of 2 used in the other experiments is therefore too low for WAN conditions; the other experiments did not count flaps and cannot be taken as evidence to the contrary. A static default must trade reaction time (low k) against noise immunity (high k), and the right point depends on network conditions that change over time.

The other two edges show the opposite side of the trade-off. On the RECOVERING→ANCHORED edge, a larger hysteresis wait strictly reduces anchored uptime under sustained critical-level noise (59.8%→0.0% as the threshold goes from 0 to 20), under the critical-level noise of Section 5.7. This follows from the design. In RECOVERING, a critical reading returns the chain directly to SOVEREIGN (the RECOVERING branch of `CalculateNextState`), so the run of healthy blocks must start again, and a longer wait is less likely to complete between two critical readings. On the SUSPICIOUS→ANCHORED edge, absorption rate rises from 0% to 100% as the threshold goes from 1 to 4, at a cost: the SUSPICIOUS timeout keeps running while the counter accumulates, so only thresholds 1–2 reached ANCHORED before the timeout forced SOVEREIGN. Any adaptive threshold must therefore stay below the SUSPICIOUS timeout, which bounds $k_{\max}$.

Adaptive hysteresis. We therefore extend the backoff already used on RECOVERING to the ANCHORED↔SUSPICIOUS boundary. The effective threshold is $k_{\text{eff}} = \min(k_{\max}, k_0 \cdot 2^c)$, where the flap counter c is incremented at every ANCHORED→SUSPICIOUS→ANCHORED round trip and decremented after every W consecutive flap-free blocks. k_{eff} depends only on committed state, so it remains deterministic across validators. It starts at the responsive value k_0 and rises only when the network proves noisy, which also raises the cost of the repeated-trigger attack discussed in Section 7. [PENDING: implement k_{eff} (TLA+ and Go), re-run the live down-edge sweep, and compare flaps,

uptime, and detection delay against static $k \in \{2, 6\}$.]

Appendix D complements these results by measuring how deep a real Bitcoin reorganization must be before the FSM reacts.

6.4. RQ4: Cost

Table 13: Overhead of the Extended Consensus Object vs. Vanilla CometBFT (20 tx/s, n=5 repeats, mean ± 95% CI)

Metric	**Vanilla**	**Engram**
Proposal overhead (steady state)	0 B	+228 B
Proposal validation CPU	—	18.25 μs
Blocks/s	0.953 ± 0.007	0.743 ± 0.008
Steady blocks/s (1/mean interval)	1.000 ± 0.000	0.993 ± 0.023
Gap fraction	0.046 ± 0.006	0.251 ± 0.016
p95 block interval	1.017 s	1.021 s
Tx accepted to mempool/s	20.00 ± 0.00	19.99 ± 0.00
Rounds per height	1.000	1.000
Proposals rejected / height	0	0
Nil precommits / height	0	0
Block interval, mean (p50 / p95)	1.034 (1.032 / 1.042) s	1.271 (1.257 / 1.342) s
Committed tx/s	20.00 ± 0.00	20.00 ± 0.00
Tx commit latency (p50 / p95)	0.67 / 1.16 s	1.03 / 1.64 s
Mempool backlog (mean / max)	8.2 / 24	15.7 / 46
Tx per block (mean)	17.6	21.8

Blocks/s, steady blocks/s, and gap fraction are defined in Section 5.6; each is a mean over n=5 runs of 60 s. The rows below the rule come from a separate n=5 pass at 20 tx/s that scans *every* committed height (commit round, nil precommits, block time) instead of polling; nil prevotes are not stored on chain, and rejected proposals (distinct height/round/reason in the validators' logs) are the proxy for why a validator would prevote nil. Transaction counts exclude the health certificate in `Txs[0]`. Latency runs from submission to the first 0.25 s poll that sees the transaction in a block. The vanilla arm is one local process, so its round and nil-vote rows are trivially 1 and 0.

Where the cost is not. The extended object adds a near-constant +228 bytes per proposal (live: 247–270 bytes across states, 100% of 376 sampled blocks), and the one proof-carrying block observed was 14.9 KB, consistent with the 14.7 KB proof. Full proposal validation costs 18.25 μs per steady-state block. Neither the byte overhead nor the validation time is large enough to explain a lower block rate at a ∼1 s block interval.

Where the cost is. Engram's block rate is lower than vanilla's (0.743 against 0.953 blocks/s in the polled runs above; 0.788 against 0.968 when every height is scanned). The scan identifies the cause by elimination. All 276 scanned Engram heights committed in the first round, none carried a nil precommit, and no proposal was rejected, so extra consensus rounds, `ProcessProposal` rejections and nil votes are ruled out. The interval distribution is also not a few long stalls: Engram's blocks arrive every 1.27 s (p50 1.257, p95 1.342) and only 0.8% are slower than 1.5 times the median, against 1.03 s for vanilla. Each height is therefore about 0.24 s longer, a constant per-height cost. The earlier reading that the gap fraction signals extra rounds was an artefact of polling, which cannot resolve intervals this close. The extra time sits on the critical path of every height, and the vote-extension step is the natural candidate: `ExtendVote` reads the live Bitcoin and Celestia sensors and each validator signs and gossips one more message. We did not isolate it, for example by replacing the extension with a no-op, so we state the constant, not its source.

Load. Between 5 and 100 tx/s, both modes accepted every submitted transaction into the mempool

and the p95 block interval stayed at 1.01–1.02 s. Mempool acceptance is not throughput; committed throughput and its ceiling are measured separately (Table 13, lower rows).

7. Discussion

7.1. What the Evidence Establishes

The formal and empirical results cover complementary ground. The TLA+ results establish, per layer and within bounded configurations, that the FSM's safety invariants hold and that removing any of four defensive mechanisms reproduces the violation it targets; Lemma 1 establishes that the certified health input to the FSM is bounded by honest observations. The live evaluation establishes that the composed implementation behaves as specified under real peripheral failures. Because the cross-layer refinement is open, the whole-system claim rests on the empirical side, and single-trial live results (composite fault, seven-scenario sweep, P4, reorg depth) show that correct behavior occurs rather than characterizing its distribution. The repeated experiments – the hysteresis sweeps, the n=5 overhead benchmark, and the 2,000-trial detector study – do characterize distributions.

7.2. Security Implications of the Fallback

The fallback trades cross-chain withdrawals for in-chain liveness. We discuss the trade along three axes: liveness inside the chain, safety of cross-chain assets, and the cost of griefing.

Liveness inside the chain. In SOVEREIGN, block production, in-chain transactions, application logic, and the forced-transaction queue continue. Cross-chain withdrawals stop. A hard-gated chain under the same degradation commits no blocks at all (Section 6.2). [PENDING: blocks committed per outage window for Engram versus the hard-gated baseline (BTC suspension, DA outage, both).]

Safety of cross-chain assets. The prototype has no bridge and no real assets: the application mounts neither `x/auth` nor `x/bank`, transactions are unsigned envelopes, and a withdrawal is recognized by the substring `TX_WITHDRAWAL` in the transaction bytes. The circuit breaker is therefore evaluated as a policy: it shows that the decision to admit or reject withdrawals is agreed by all validators and applied identically, not that any economic loss is prevented.

Cost of griefing. An adversary that degrades the peripheral view of *all* honest validators – for example by delaying their Bitcoin connectivity – can drive the chain into SOVEREIGN and freeze withdrawals for as long as the degradation lasts (P4, Section 6.1). This is a new attack surface relative to a chain that ignores its periphery, but not relative to a hard-gated chain: the same attacker halts the hard-gated chain entirely, whereas Engram freezes only withdrawals. Lemma 1 prevents a minority of validators from causing the freeze, so the attacker must degrade the peripheral links of at least $2f+1$ validators; in a design without a quorum-backed health input, one node's view suffices. The freeze is the intended response to an unverifiable periphery, not a safety violation. We do not quantify its economic cost to users or the attacker's cost of sustaining it.

Unexpected results. Three results need a causal account.

- *Block rate.* Engram's rate is 22% below vanilla's (Section 6.4). The per-height scan rules out extra rounds, rejections and nil votes: they are absent. What remains is a constant 0.24 s per height. The vote-extension path (a live sensor read in `ExtendVote` and one more signed message per validator) sits on every height's critical path and is the natural candidate, but we did not isolate it, for example by timing `ExtendVote` against a no-op extension.

- *Hysteresis threshold.* The default threshold of 2 flapped under wide-area jitter, and only thresholds ≥ 6 were clean (Section 6.3). We attribute this to latency variance in the Bitcoin sensor crossing the threshold in consecutive samples; a run with the jitter profile removed would test this.
- *Reorg depth.* Reorgs of depth 3–5 orphaned confirmed blocks without a reaction, while depth 15 escalated through the suspicious-duration bound (Appendix D). Whether the anchor-reverification flag was ever set is not established, because the per-validator sensor value was not captured; logging it and sweeping depths 6–14 would locate the boundary.

Repeated triggering of Suspicious. Backoff on RECOVERING already raises the price of forcing repeated failed recoveries. Without the adaptive threshold of Section 6.3, the ANCHORED↔SUSPICIOUS boundary has no such protection beyond the bounded forced-transaction queue; the adaptive threshold closes this asymmetry.

Core versus peripheral network. Engram tolerates degradation of the peripheral layers (Bitcoin, Celestia, and the peripheral or partial P2P view) only while the core consensus network keeps $2f+1$ voting power mutually connected. If CometBFT's own quorum is partitioned, no FSM can restore liveness; this is the standard BFT assumption, not a limitation specific to Engram, and none of our experiments cross it.

Proposer bias within the honest range. When honest observations straddle a threshold, a proposer can choose which honest value the certificate selects (Section 3.4). This cannot produce a transition that no honest view supports, but a sequence of Byzantine proposers can bias the timing of transitions. A persistent split straddling the $2f+1$ boundary (e.g. three versus four validators) has not been tested live; the RQ4 scan shows no extra rounds in the unfaulted case, and the split case itself remains untested.

History divergence at re-anchoring. During SOVEREIGN, the ordering of blocks rests on the validator quorum alone. A coalition exceeding f could produce a SOVEREIGN history that differs from one it would have produced under Bitcoin timestamping; the recovery proof certifies the history's internal legality and quorum signatures, not its uniqueness beyond the f bound. The withdrawal lock limits what such a history can extract.

Data availability of the Sovereign interval. During SOVEREIGN, the interval's block data is held by the validators' own block stores, so its recoverability depends on those validators until it is published. The backfill gate of Section 3.9 makes publication a precondition for returning to ANCHORED.

7.3. Limitations

- *Single physical host.* Absolute performance figures (RQ4) and timing results are proxies for a geographically distributed deployment; wide-area conditions are emulated.
- *Accelerated Bitcoin.* Wall-clock durations show the ordering of recovery steps, not mainnet latencies; height-based thresholds are unaffected.
- *DA trust.* Each validator trusts its own bridge rather than sampling, and the backfill gate rests on a quorum-reported contiguous height, not on a proof of retrievability (Section 5.9).
- *Proof gate.* The recovery-validity flag holds if a verified proof was submitted within the permitted unproven tail of the tracked tip, or if a heuristic holds: the last submitted Bitcoin

checkpoint has reached the anchored depth. A protocol parameter disables the heuristic, so that only a verified proof satisfies the gate. [PENDING: state which reported recovery runs used proof-only recovery.]

- *Byzantine scale.* The live attacks reported so far involve one Byzantine actor at a time, and A6 and A1/A2 ran at n=4; the f=2 collusion suite (Section 6.1) addresses this.
- *Formal scope.* Per-layer bounded model checking is complete; the cross-layer refinement is open. The health certificate is modelled with Boolean observations in the server layer, so its checks fall under the open refinement; numeric aggregation is covered by Lemma 1 only.
- *Load range.* Load was tested up to 100 tx/s; behavior at higher rates, where per-block overhead could interact with mempool and gossip costs, is not characterized.

7.4. Threats to Validity

Internal validity. Host sharing can confound cost measurements. We mitigate it by reporting repeated measurements with confidence intervals and by comparing Engram and vanilla on the same host under the same load; the host still affects absolute values.

External validity. Results at seven validators on one host need not extrapolate to larger or geographically distributed validator sets. We interpret them as evidence of correct behavior and of the source of the overhead, not as production performance figures.

Construct validity. AppHash agreement and height progression are the conventional operational proxies for safety and liveness in CometBFT-family systems. AppHash agreement alone cannot detect an illegal transition agreed on by all validators; the semantic replay check of Section 6.1 addresses this gap.

8. Conclusion

Modular chains that make settlement and DA receipts part of block validity halt when those layers fail. Engram keeps such a chain live by suspending the external dependency together with the operations whose safety relied on it, and by making that suspension part of deterministic replicated state. It leaves CometBFT's agreement core unchanged and extends the consensus object with a quorum-backed health certificate, whose aggregate is bounded by honest observations, and a four-state FSM evaluated deterministically on that certificate. When the certificate reports a peripheral layer as unhealthy, the corresponding receipt requirement is relaxed and cross-chain withdrawals are locked. When health returns, recovery hysteresis, DA backfill of the SOVEREIGN interval, and a validator-verified recovery proof over that interval gate the return to ANCHORED.

Per-layer bounded model checking found no violations (the largest run, the FSM safety check, explored 3.39 million distinct states), and a five-mechanism ablation reproduced a distinct counterexample for each removed mechanism. On a seven-validator testbed with live Bitcoin and Celestia networks, the chain kept committing blocks under a layered composite fault and completed a full recovery cycle with complete AppHash agreement. The extended object costs +228 bytes and 18.25 μs per proposal. Its lower block rate is not caused by extra consensus rounds: every height commits in the first round with no nil precommit or rejected proposal, and each block takes a near-constant 0.24 s longer than vanilla's (1.27 s against 1.03 s).

Future Work

Three directions remain beyond the items marked in this paper. First, closing the cross-layer refinement obligation, and extending the certificate model from Boolean to numeric observations, would extend the formal results to the composed system; deductive methods [44, 71] are a natural complement where bounded model checking does not scale. Second, replacing the fixed-arity aggregator with incremental recursion would make the recovery artifact constant-size for intervals of any length, and a live re-anchoring through an aggregate proof would extend the offline aggregation measurements to the full testbed. Third, replacing single-bridge DA trust with sampling across independent peers would let the DA sensor detect partial withholding rather than only outages.

Artifact Availability

[PENDING: repository URL, license, and availability date.] The implementation, TLA+ specifications, and experiment scripts are publicly available.

References

[1] M. Al-Bassam, "LazyLedger: A Distributed Data Availability Ledger With Client-Side Smart Contracts," arXiv:1905.09274, 2019, https://arxiv.org/abs/1905.09274.

[2] E. Buchman, "Tendermint: Byzantine Fault Tolerance in the Age of Blockchains," Ph.D. dissertation, University of Guelph, 2016, https://atrium.lib.uoguelph.ca/items/6c1ad7d4-7e5c-4f7f-b3d4-3f60d4fda1f5.

[3] E. Buchman, J. Kwon, and Z. Milosevic, "The Latest Gossip on BFT Consensus," arXiv:1807.04938, 2018, https://arxiv.org/abs/1807.04938.

[4] J. Neu, E. N. Tas, and D. Tse, "Ebb-and-Flow Protocols: A Resolution of the Availability-Finality Dilemma," in *Proceedings of the 2021 IEEE Symposium on Security and Privacy (S&P)*, 2021, pp. 446–465.

[5] V. Buterin, D. Hernandez, T. Kamphefner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, "Combining GHOST and Casper," arXiv:2003.03052, 2020.

[6] F. Gomes Figueira, M. Derka, C. L. Chiu, and J. Gorzny, "A Practical Rollup Escape Hatch Design," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2025)*, 2025, https://arxiv.org/abs/2503.23986.

[7] Offchain Labs, "Inside AnyTrust," Arbitrum documentation, https://docs.arbitrum.io/inside-anytrust, accessed 2026-10-02.

[8] Optimism contributors, "Alt-DA Mode Specification," OP Stack experimental specification, https://github.com/ethereum-optimism/specs/blob/main/specs/experimental/alt-da.md, accessed 2026-10-03.

[9] Cosmos SDK contributors, "ADR 064: ABCI 2.0 Integration (Phase II)," Cosmos SDK architecture decision record, https://github.com/cosmos/cosmos-sdk/blob/main/docs/architecture/adr-064-abci-2.0.md, accessed 2026-10-03.

[10] Skip Protocol and dYdX contributors, "Slinky: A General-Purpose Oracle Leveraging ABCI++," Software repository (continued as skip-mev/connect), https://github.com/dydxprotocol/slinky, accessed 2026-10-03.

[11] N. C. Mendonça, C. M. Aderaldo, J. Cámara, and D. Garlan, "Model-Based Analysis of Microservice Resiliency Patterns," in *Proceedings of the 2020 IEEE International Conference on Software Architecture (ICSA)*, 2020, pp. 114–124.

[12] R. Pass and E. Shi, "Thunderella: Blockchains with Optimistic Instant Confirmation," in *Advances in Cryptology – EUROCRYPT 2018*. Springer, 2018.

[13] P.-L. Aublin, R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, "The Next 700 BFT Protocols," *ACM Transactions on Computer Systems*, vol. 32, no. 4, pp. 1–45, 2015.

[14] N. Giridharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, "Autobahn: Seamless High Speed BFT," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles (SOSP 2024)*, 2024, pp. 1–23.

[15] R. Pass and E. Shi, "The Sleepy Model of Consensus," in *Advances in Cryptology – ASIACRYPT 2017*, ser. Lecture Notes in Computer Science, vol. 10625, 2017, pp. 380–409.

[16] V. Buterin and V. Griffith, "Casper the Friendly Finality Gadget," arXiv:1710.09437, 2017.

[17] T. D. Chandra and S. Toueg, "Unreliable Failure Detectors for Reliable Distributed Systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.

[18] W. Chen, S. Toueg, and M. K. Aguilera, "On the Quality of Service of Failure Detectors," *IEEE Transactions on Computers*, vol. 51, no. 5, pp. 561–580, 2002.

[19] N. Hayashibara, X. Défago, R. Yared, and T. Katayama, "The φ Accrual Failure Detector," in *Proceedings of the 23rd IEEE International Symposium on Reliable Distributed Systems (SRDS)*, 2004, pp. 66–78.

[20] A. Doudou, B. Garbinato, R. Guerraoui, and A. Schiper, "Muteness Failure Detectors: Specification and Implementation," in *Dependable Computing – EDCC-3*, 1999, pp. 71–87.

[21] D. Dolev, N. A. Lynch, S. S. Pinter, E. W. Stark, and W. E. Weihl, "Reaching Approximate Agreement in the Presence of Faults," *Journal of the ACM*, vol. 33, no. 3, pp. 499–516, 1986.

[22] D. Stolz and R. Wattenhofer, "Byzantine Agreement with Median Validity," in *19th International Conference on Principles of Distributed Systems (OPODIS 2015)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 46, 2015, pp. 22:1–22:14.

[23] F. Zhang, E. Cecchetti, K. Croman, A. Juels, and E. Shi, "Town Crier: An Authenticated Data Feed for Smart Contracts," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016, pp. 270–282.

[24] S. Eskandari, M. Salehi, W. C. Gu, and J. Clark, "SoK: Oracles from the Ground Truth to Market Manipulation," in *Proceedings of the 3rd ACM Conference on Advances in Financial Technologies (AFT '21)*, 2021, pp. 127–141.

[25] Y. Xiao, N. Zhang, W. Lou, and Y. T. Hou, "A Decentralized Truth Discovery Approach to the Blockchain Oracle Problem," in *Proceedings of IEEE INFOCOM 2023*, 2023, pp. 1–10.

[26] Y. Yao, Y. Shi, G. Tian, M. Miao, and W. Susilo, "PSCBO: A Provably Secure Consensus-Based Blockchain Oracle," *Computer Standards & Interfaces*, vol. 91, 2025.

[27] M. Al-Bassam, A. Sonnino, V. Buterin, and I. Khoffi, "Fraud and Data Availability Proofs: Detecting Invalid Blocks in Light Clients," in *Financial Cryptography and Data Security (FC 2021)*, ser. Lecture Notes in Computer Science, vol. 12675, 2021, pp. 279–298.

[28] D. Pham Minh, H. Vuong Huu, and D. A. Tran, "Efficient Data Availability Sampling via Coded Distributed Arrays," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, https://arxiv.org/abs/2606.16200.

[29] D. Karakostas and A. Kiayias, "Securing Proof-of-Work Ledgers via Checkpointing," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2021)*, 2021.

[30] E. N. Tas, D. Tse, F. Yu, and S. Kannan, "Babylon: Reusing Bitcoin Mining to Enhance Proof-of-Stake Security," arXiv:2201.07946, 2022, https://arxiv.org/abs/2201.07946.

[31] S. Azouvi and M. Vukolić, "Pikachu: Securing PoS Blockchains from Long-Range Attacks by Checkpointing into Bitcoin PoW using Taproot," in *Proceedings of the 2022 ACM Workshop on Developments in Consensus (ConsensusDay '22)*, 2022, pp. 53–65.

[32] E. N. Tas, D. Tse, F. Gai, S. Kannan, M. A. Maddah-Ali, and F. Yu, "Bitcoin-Enhanced Proof-of-Stake Security: Possibilities and Impossibilities," in *Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P)*, 2023, pp. 126–145.

[33] M. Capretto, M. Ceresa, A. Fernández Anta, P. Moreno-Sanchez, and C. Sánchez, "A Secure Sequencer and Data Availability Committee for Rollups," in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2025.

[34] ——, "A Decentralized Sequencer and Data Availability Committee for Rollups Using Set Consensus," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2026)*, 2026, pp. 1–9.

[35] T. Xie, J. Zhang, Z. Cheng, F. Zhang, Y. Zhang, Y. Jia, D. Boneh, and D. Song, "zkBridge: Trustless Cross-chain Bridges Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 3003–3017.

[36] P. Vesely, K. Gurkan, M. Straka, A. Gabizon, P. Jovanovic, G. Konstantopoulos, A. Oines, M. Olszewski, and E. Tromer, "Plumo: An Ultralight Blockchain Client," in *Financial Cryptography and Data Security (FC 2022)*, 2022, pp. 597–614.

[37] J. Bonneau, I. Meckler, V. Rao, and E. Shapiro, "Coda: Decentralized Cryptocurrency at Scale," Cryptology ePrint Archive, Paper 2020/352, 2020, later renamed Mina.

[38] B. Bünz, L. Kiffer, L. Luu, and M. Zamani, "FlyClient: Super-Light Clients for Cryptocurrencies," in *Proceedings of the 2020 IEEE Symposium on Security and Privacy (S&P)*, 2020, pp. 928–946.

[39] N. Bitansky, R. Canetti, A. Chiesa, and E. Tromer, "Recursive Composition and Bootstrapping for SNARKs and Proof-Carrying Data," in *Proceedings of the 45th Annual ACM Symposium on Theory of Computing (STOC)*, 2013, pp. 111–120.

[40] S. Bowe, J. Grigg, and D. Hopwood, "Recursive Proof Composition without a Trusted Setup," Cryptology ePrint Archive, Paper 2019/1021, 2019.

[41] A. Kothapalli, S. Setty, and I. Tzialla, "Nova: Recursive Zero-Knowledge Arguments from Folding Schemes," in *Advances in Cryptology – CRYPTO 2022*, ser. Lecture Notes in Computer Science, vol. 13510, 2022, pp. 359–388.

[42] I. Konnov, J. Kukovec, and T.-H. Tran, "TLA+ Model Checking Made Symbolic," *Proceedings of the ACM on Programming Languages*, vol. 3, no. OOPSLA, 2019.

[43] S. Braithwaite, E. Buchman, I. Konnov, Z. Milosevic, I. Stoilkovska, J. Widder, and A. Zamfir, "Formal Specification and Model Checking of the Tendermint Blockchain Synchronization Protocol (Short Paper)," in *2nd Workshop on Formal Methods for Blockchains (FMBC 2020)*, ser. OpenAccess Series in Informatics (OASIcs), vol. 84, 2020.

[44] O. Padon, K. L. McMillan, A. Panda, M. Sagiv, and S. Shoham, "Ivy: Safety Verification by Interactive Generalization," in *Proceedings of the 37th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2016, pp. 614–630.

[45] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "IronFleet: Proving Practical Distributed Systems Correct," in *Proceedings of the 25th Symposium on Operating Systems Principles (SOSP)*, 2015, pp. 1–17.

[46] J. R. Wilcox, D. Woos, P. Panchekha, Z. Tatlock, X. Wang, M. D. Ernst, and T. Anderson, "Verdi: A Framework for Implementing and Formally Verifying Distributed Systems," in *Proceedings of the 36th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2015, pp. 357–368.

[47] L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, W. Honoré, and Z. Shao, "LiDO: Linearizable Byzantine Distributed Objects with Refinement-Based Liveness Proofs," *Proceedings of the ACM on Programming Languages*, vol. 8, no. PLDI, 2024.

[48] S. Bano, A. Sonnino, A. Chursin, D. Perelman, Z. Li, A. Ching, and D. Malkhi, "Twins: BFT Systems Made Robust," in *25th International Conference on Principles of Distributed Systems (OPODIS 2021)*, ser. LIPIcs, vol. 217, 2022, pp. 7:1–7:29.

[49] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, and C. Rosenthal, "Chaos Engineering," *IEEE Software*, vol. 33, no. 3, pp. 35–41, 2016.

[50] M. Castro and B. Liskov, "Practical Byzantine Fault Tolerance," in *Proceedings of the Third Symposium on Operating Systems Design and Implementation (OSDI '99)*, 1999, pp. 173–186.

[51] C. Dwork, N. Lynch, and L. Stockmeyer, "Consensus in the Presence of Partial Synchrony," *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.

[52] M. Yin, D. Malkhi, M. K. Reiter, G. Golan-Gueta, and I. Abraham, "HotStuff: BFT Consensus with Linearity and Responsiveness," in *Proceedings of the 2019 ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.

[53] G. Danezis, L. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, "Narwhal and Tusk: A DAG-based Mempool and Efficient BFT Consensus," in *Proceedings of the Seventeenth European Conference on Computer Systems (EuroSys '22)*, 2022, pp. 34–50.

[54] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, "Bullshark: DAG BFT Protocols Made Practical," in *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.

[55] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and Efficient Asynchronous Broadcast Protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. Lecture Notes in Computer Science, vol. 2139. Springer, 2001, pp. 524–541.

[56] P. Sheng, G. Wang, K. Nayak, S. Kannan, and P. Viswanath, "BFT Protocol Forensics," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2021, pp. 1722–1743.

[57] W. Honoré, L. Qiu, Y. Kim, J.-Y. Shin, J. Kim, and Z. Shao, "AdoB: Bridging Benign and Byzantine Consensus with Atomic Distributed Objects," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 419–448, 2024.

[58] W. Honoré, J.-Y. Shin, J. Kim, and Z. Shao, "Adore: Atomic Distributed Objects with Certified Reconfiguration," in *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '22)*, 2022, pp. 379–394.

[59] S. Gilbert and N. Lynch, "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[60] J. R. Douceur, "The Sybil Attack," in *Peer-to-Peer Systems (IPTPS 2002)*, ser. Lecture Notes in Computer Science, vol. 2429. Springer, 2002, pp. 251–260.

[61] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg, "Eclipse Attacks on Bitcoin's Peer-to-Peer Network," in *Proceedings of the 24th USENIX Security Symposium*, 2015, pp. 129–144.

[62] Z. Rehman, M. A. Gregory, I. Gondal, H. Dong, and M. Ge, "Eclipse Attacks in Blockchain Networks: Detection, Prevention, and Future Directions," *IEEE Access*, vol. 13, pp. 25 918–25 933, 2025.

[63] A. Back, M. Corallo, L. Dashjr, M. Friedenbach, G. Maxwell, A. Miller, A. Poelstra, J. Timón, and P. Wuille, "Enabling Blockchain Innovations with Pegged Sidechains," Whitepaper, Blockstream, 2014.

[64] H. Kalodner, S. Goldfeder, X. Chen, S. M. Weinberg, and E. W. Felten, "Arbitrum: Scalable, Private Smart Contracts," in *Proceedings of the 27th USENIX Security Symposium*, 2018, pp. 1353–1370.

[65] L. Gudgeon, P. Moreno-Sanchez, S. Roos, P. McCorry, and A. Gervais, "SoK: Layer-Two Blockchain Protocols," in *Financial Cryptography and Data Security (FC 2020)*, ser. Lecture Notes in Computer Science, vol. 12059, 2020, pp. 201–226.

[66] "Arbitrum Orbit AnyTrust Chains," Arbitrum blog, https://blog.arbitrum.io/arbitrum-orbit-anytrust-chains/, accessed 2026-10-02.

[67] S.-S. Lee, A. Murashkin, M. Derka, and J. Gorzny, "SoK: Not Quite Water Under the Bridge: Review of Cross-Chain Bridge Hacks," in *Proceedings of the IEEE International Conference on Blockchain and Cryptocurrency (ICBC 2023)*, 2023, pp. 1–14, arXiv:2210.16209.

[68] E. Deirmentzoglou, G. Papakyriakopoulos, and C. Patsakis, "A Survey on Long-Range Attacks for Proof of Stake Protocols," *IEEE Access*, vol. 7, pp. 28 712–28 725, 2019.

[69] A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive Arguments of Knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019.

[70] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002, https://lamport.azurewebsites.net/tla/book.html.

[71] R. Lotan and S. Shoham, "Implicit Rankings for Verifying Liveness Properties in First-Order Logic," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS 2025)*, 2025.

[72] N. Bondarev, K. Ziborov, and Y. Yanovich, "Systematization of Knowledge: Formal Verification of Consensus Protocols," arXiv:2608.21935, 2026, https://arxiv.org/abs/2608.21935.

[73] Engram Protocol, "Engram Protocol: Technical Whitepaper," 2026, internal, unpublished organizational whitepaper. Appendix C sketches the adaptive-consensus / circuit-breaker concept formalized, verified, and evaluated in this paper.

[74] J. Kwon and E. Buchman, "Cosmos: A Network of Distributed Ledgers," Whitepaper, 2016, https://v1.cosmos.network/resources/whitepaper.

[75] Aztec Labs, "Noir: A Domain-Specific Language for Zero-Knowledge Proofs, and the Barretenberg Proving Backend," Software documentation, 2024, noir: https://noir-lang.org; Barretenberg: https://github.com/AztecProtocol/barretenberg.

Appendix

A. Extended Attack-Resilience Results

Table 14 reports the full ten-scenario single-actor attack suite. A1 and A2 ran on the earlier four-validator cluster (n=4); all other scenarios ran on the seven-validator testbed (n=7). The column *Byz. VP* gives the share of voting power held by the misbehaving validator: one of seven validators is 14.3%, and the attacker containers of A1 and A2 are non-validators (0%).

Table 14: Full Attack-Resilience Suite (live)

#	Attack	n	Byz. VP	Safety	Diverg.	Attack-specific signal
A1	Eclipse	4	0%	True	0	Filter holds 8/8, nil-prevote ratio 0.0 throughout
A2	Simulated multi-subnet Sybil	4	0%	True	0	Filter holds 8/8 despite 12 attackers
A3	Data withholding	7	14.3%	True	0	—
A4	Forged Bitcoin receipt	7	14.3%	True	0	—
A5	Withdrawal during SOVEREIGN	7	—	True	0	Withdrawal correctly blocked; transaction never commits
A6	Malicious proposer	7	14.3%	True	0	Forged FSM state rejected by all honest validators (Section 6.1)
A7	Censorship	7	14.3%	True	0	Censoring confirmed real (target tx omitted 9/9 proposer turns); honest reject signals stayed at 0 since the next honest proposer included the tx before the ignore-round threshold fired
A8	Combined (A6 + A1)	7	14.3%	True	0	Safety held under simultaneous overlap
A9	Double-signing	7	14.3%	True	0	6/6 honest validators detected real duplicate-vote evidence; 1-block detection latency
A10	Timeout flood	7	14.3%	True	0	Cadence held (0.770→0.706 blocks/s); rate limiter dropped 0/0/0/0/0/0 at moderate (20 msg/s) intensity

A3 and A4 were instrumented only for safety and divergence, without an attack-specific metric such as A5's blocked-withdrawal flag or A9's detection latency.

At 25× its design threshold (500 msg/s vs. a 20 msg/s per-peer cap), the A10 timeout-flood attack still held cadence (0.825→0.789 blocks/s) while the rate limiter rejected the large majority of flood traffic (28,071 of ∼30,000 attempted messages on one validator); validator CPU usage roughly tripled (2.4–3.3%→7.85–9.51%, peak 22.40%) but stayed well short of saturation, and memory usage stayed flat. No attack in Table 14 drove block production toward zero, and resource usage under the P2P swarm attacks (A1/A2) stayed modest (under 36% CPU, under 147 MiB memory per validator).

B. Backend Trade-off: Noir+Honk vs. Plonky3

This appendix compares the deployed Noir/UltraHonk recovery circuit (Section 3.9) with a STARK-based alternative, Plonky3. The comparison is necessarily partial: boundary-signature verification,

condition (e), dominates the deployed circuit's cost (1,097,274 opcodes with it versus 6,143 without, a factor of ∼180), and no Plonky3 configuration we built implements it.

Table 15: Deployed Circuit: Constant-Cost Scaling (Noir+Honk, Poseidon2)

Interval count	**Prove time**	**Verify time**	**Proof size**
1	38.3 s	25 ms	14,656 B
16	41.4 s	23 ms	14,656 B
256 ($N_{\max}$)	40.4 s	24 ms	14,656 B

Table 15 confirms that proving time, verification time, and proof size are independent of the number of real headers up to the compiled capacity $N_{\max}$=256. Verification by the validators' embedded WebAssembly verifier costs ∼26 ms per proof. An interval of L blocks requires $\lceil L/256 \rceil$ leaf proofs when submitted one by one, or a single aggregate proof up to 1024 blocks (Appendix C).

Table 16: Backend Comparison, Statement Coverage

Config.	(a)	(b)	(c)	(d)	(e)
Noir+Honk (deployed)	✓	✓	✓	✓	✓
Plonky3, primitive-only	–	–	–	–	–
Plonky3, fuller statement	✓	✓	✓	✓	–

(a) chain continuity, (b) FSM/withdrawal-lock policy legality, (c) circuit-breaker adherence, (d) state-root binding, (e) boundary ed25519-signature verification.

Table 16 makes the coverage gap explicit. Under a primitive-only comparison (256 chained Poseidon2 permutations, no FSM or signature logic, i.e. neither configuration implements condition (e)), Plonky3 produces a substantially larger proof (1,278,939 B vs. 14,656 B, ∼87×) but proves faster (0.044 s vs. 0.684 s, ∼15.5×) with a transparent, post-quantum-secure setup versus the Noir/Honk backend's trusted KZG setup. Under a fuller-statement comparison that adds FSM legality and root binding (but still omits condition (e)), Plonky3 proves in 0.024–0.096 s with proof sizes of 84–124 KB across 2–256 headers. Neither comparison supports a general speed claim for Plonky3, because both omit the component that dominates the deployed circuit's cost. These numbers characterize primitive-level backend trade-offs only.

C. Chunk Aggregation

This appendix measures the aggregator of Section 3.9 against submitting the chunk proofs one by one. The measurements were taken offline on a 10-core, 32 GiB macOS workstation, not on Testbed A, with nargo 1.0.0-beta.22 and `bb` 5.0.0-nightly.20260522. The chain is synthetic: it has one deterministic header sequence with real ed25519 precommits and one validator set, so that consecutive chunks chain through their roots and heights. Aggregate proving and verification are averaged over five trials per interval length; each leaf proof was timed once. Absolute times are not comparable to Table 15, which was measured on a different host.

The aggregator verifies the chunk proofs in-circuit and checks that chunk i starts at the root and height where chunk $i-1$ ended, under the same validator-set hash. It exposes the same public inputs as a single chunk, so an aggregate proof stands in for a leaf proof, and its capacity K=4 is fixed at compile time. Unbounded incremental recursion would require the step circuit to receive its own verification-key hash as a public input, which we did not implement. Recursion is applied at the

Table 17: Chunk Aggregation vs. Submitting Chunk Proofs One by One (K=4)

	One by one			Aggregate		
L	Prove	Bytes	Verify	Prove	Bytes	Verify
256	31.5 s	14,656	19.8 ms	57.2 s	14,656	20.2 ms
512	60.5 s	29,312	39.5 ms	89.9 s	14,656	20.2 ms
1024	120.1 s	58,624	79.1 ms	146.7 s	14,656	20.2 ms

Verify is the validators' embedded WebAssembly verifier, measured per proof (30 iterations, three runs); the one-by-one column is n times the leaf cost. Aggregate prove includes the leaf proofs. At L=256 a single leaf proof is submitted and the aggregate row is shown for the overhead only.

chunk level because in-circuit verification of one UltraHonk proof costs about 0.7 million gates, whereas hashing one header costs about 12 opcodes: recursion over per-header proofs measured 650–1,200× slower than direct hashing at matched sizes.

Table 17 shows that submitted bytes and verification time of the aggregate do not depend on L up to $K \cdot 256 = 1024$, while the one-by-one cost grows linearly. The price is one extra proof: the aggregator circuit has 3,002,299 gates and proves in 25.7–29.5 s using about 5.2 GB, against 3,234,226 gates, 30.0–31.5 s, and 5.7–5.9 GB for a leaf. Total proving time rises by 49% at L=512 and 22% at L=1024. Chunk proofs were produced sequentially; proving them in parallel was not measured. Every chunk boundary height needs a round-zero commit, so a round skip at an interior boundary prevents building that chunk's witness.

The aggregator rejects each of eleven malformed witnesses: reversed or repeated chunks, a wrong total count, final root, first height, or validator-set hash, a height gap between chunks, a different validator set in the second chunk, an extension through padding slots, and a chunk count outside $[1, K]$. The valid two-chunk witness is accepted, and a 512-header aggregate passes every `SubmitRecoveryProof` binding against fixture chain state. A live re-anchoring through an aggregate proof on the Docker testbed was not run.

D. Bitcoin Reorg Depth Reaction

Section 4.2 states a threat-model boundary – safety is modeled given Bitcoin reorganizations no deeper than a configurable finality depth $K_{\text{DEEP_FINALITY}}$ – as a precondition of the formal result. This appendix tests the implementation at and beyond that boundary with forced Bitcoin reorganizations, each confirmed from the resulting block hashes, at five depths in one seven-validator run: {1, 2, 3, 5, 15}, with $K_{\text{DEEP_FINALITY}}$=2 and a suspicious-state threshold of 5.

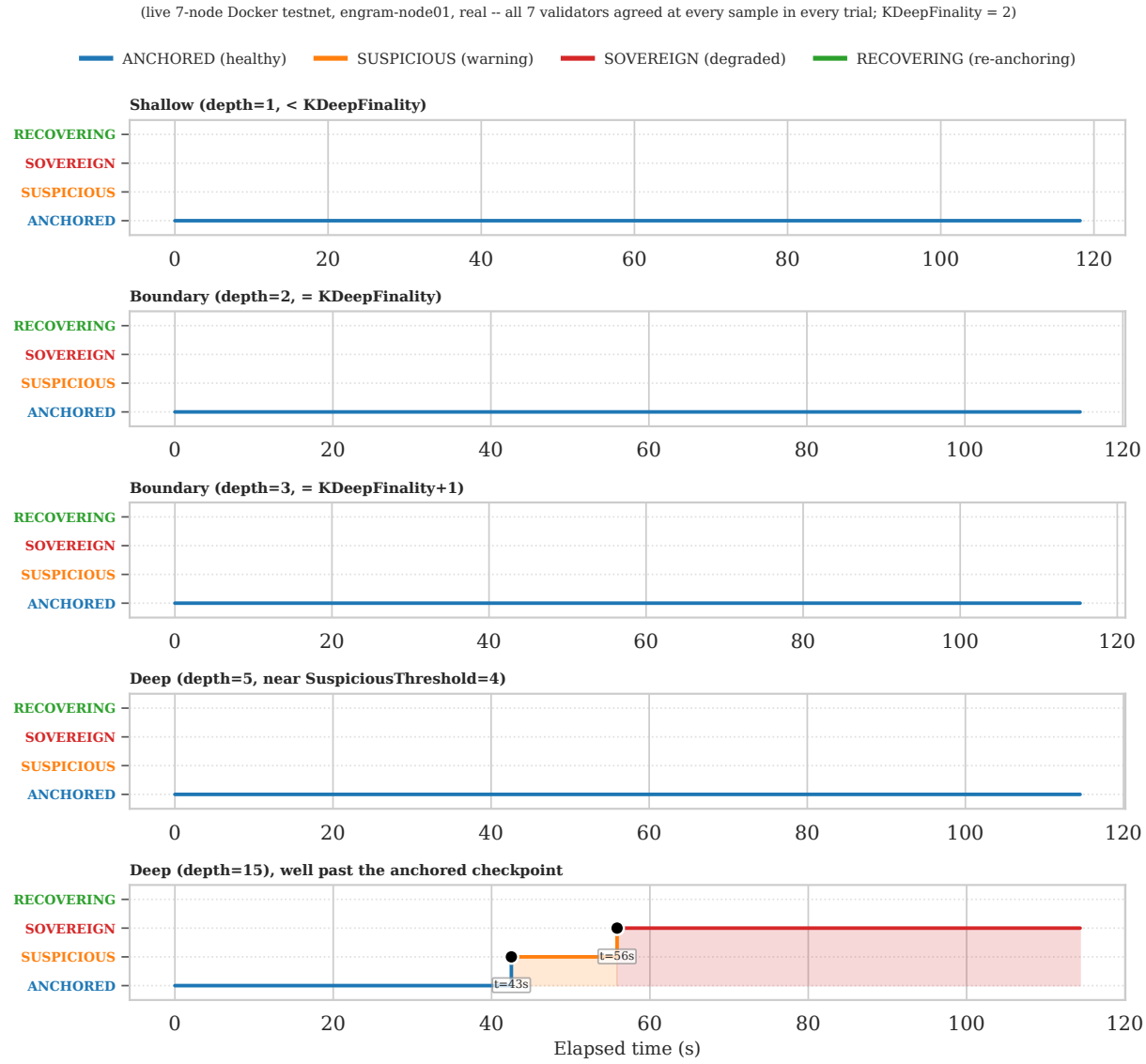


Figure 8: FSM reaction to Bitcoin reorgs at five depths, one trial per depth, seven-validator testbed. All 7 validators agree at every one of 181 polled rounds per trial (1,267 samples total, 0 RPC errors).

Table 18 summarizes the result. Depths 1–5 all orphan real, independently-confirmed blocks, yet none of them trip the FSM within the observation window: all 7 validators stay ANCHORED throughout. At depth 15, all 7 validators transition ANCHORED→SUSPICIOUS at t=43 s and →SOVEREIGN at t=56 s, in lockstep. This path rules out a direct critical-condition jump: the FSM's ANCHORED branch bypasses SUSPICIOUS entirely when the committed certificate's anchor-reverification flag is set, but the observed path instead passes through SUSPICIOUS, and the suspicious-duration counter reached 23 of the 24-block escalation bound (MaxSuspiciousTime) in the last sample before the SOVEREIGN transition. The depth-15 reaction is attributable to this duration-based escalation, not to anchor re-verification failing.

Table 18: FSM Reaction to Real Bitcoin Reorgs by Depth

Depth	**Reorg confirmed?**	**FSM reaction**
1 (below K_{DEEP})	N/A (too shallow)	Stays ANCHORED
2 (= K_{DEEP})	Yes	Stays ANCHORED
3 (= K_{DEEP}+1)	Yes	Stays ANCHORED
5	Yes	Stays ANCHORED (within window)
15 (past checkpoint)	Yes	SUSPICIOUS (43s) → SOVEREIGN (56s)

The crossover between depth 5 (no reaction) and depth 15 (reaction) was not swept at finer granularity. Whether anchor re-verification ever set the anchor-reverification flag during this run is not established by the available logs – the per-validator sensor value was not captured, only the committed FSM state and suspicious-duration counter. The data that is available shows the depth-15 escalation came from the suspicious-duration bound, not from a direct critical-condition jump.

E. Network Emulation Parameters

Table 19 lists every `tc netem` profile, applied through Pumba. No correlation or distribution option is set, so delay jitter is uniform and loss is independent per packet (the `netem` defaults). Profiles that name no interface use Pumba's default interface (`eth0`), which carries Bitcoin, Celestia, and RPC traffic rather than the dedicated pairwise validator links. Only the S4 scenario and the relay-latency attack (P4) of Section 6 target the validator links explicitly.

Table 19: Network Emulation Profiles (Testbed A)

Profile	**Effect and target**	**Duration**	**Used by**
`chaos-delay`	100 ms delay, 20 ms jitter, all seven validators	5 min	baseline
`chaos-loss`	5% loss, validators 1 and 2	2 min	composite
`chaos-eclipse`	100% loss, validator 1	3 min	S5
`chaos-btc-delay`	500 ms delay, 100 ms jitter, one `bitcoind`	2 min	composite
`chaos-wan-latency`	Per-validator delay and jitter, Table 20	10 min	WAN baseline
`chaos-wan-loss`	Per-validator loss, Table 20	10 min	WAN variant
S4 eclipse	300 ms delay, 50 ms jitter on the six validator links of validator 1	2 min	S4
P4 relay latency	350 ms delay, 50 ms jitter on all P2P interfaces of one validator	244 s	P4

Each validator receives one delay value on its egress, so the profile is a per-validator offset rather than a pairwise round-trip-time matrix. A pairwise matrix between regions is measured only on Testbed B. [PENDING: measure and tabulate the inter-region round-trip times of Testbed B.]

Table 20: WAN Baseline Profiles Per Validator (one profile of seven emulated regions)

Validator	1	2	3	4	5	6	7
Delay (ms)	15	70	140	45	100	30	180
Jitter (ms)	3	15	25	10	20	6	30
Loss (%, loss variant)	1	3	6	2	4	1	5